

# RAPPORT DE PROJET

---

## BrainOps

*Système de Prédiction du Risque*

*de Décrochage Universitaire*

*Mise en œuvre d'une pipeline DataOps et MLOps*

### Réalisé par :

Fatima Zahra ABLOU

Hasnaa ECHCHATBI

Soukaina GRANDI

Wijdane BASSIRY

Yousra OUATILE

Doaa ROMADI

Hiba MHADA

Hajar FOUZI

### Encadré par :

Pr. AITDAOUD Mohammed

**Année universitaire : 2025–2026**

# Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous remercions tout particulièrement **Mr AITDAOUD Mohammed**, pour sa disponibilité, ses conseils avisés et son accompagnement tout au long de ce projet. Ses orientations méthodologiques et son exigence nous ont permis de progresser et de mener ce travail à son terme dans les meilleures conditions.

Nos remerciements s'adressent également à l'ensemble du corps professoral et administratif de la **Faculté des Sciences Ben M'sik**, pour la qualité de la formation dispensée et l'environnement propice au développement de nos compétences techniques et méthodologiques.

Nous souhaitons aussi remercier chaleureusement chaque membre de l'équipe **BrainOps** pour son engagement, son sérieux et son esprit de collaboration tout au long de ce projet.

La complémentarité de nos rôles — de la gestion de produit à l'ingénierie des données, en passant par le Machine Learning et le déploiement — a été la clé de la réussite de ce travail collectif.

*L'équipe BrainOps*

# Résumé

Le décrochage universitaire constitue aujourd’hui un enjeu majeur pour les établissements d’enseignement supérieur, tant sur le plan pédagogique qu’institutionnel. La généralisation des environnements numériques d’apprentissage (LMS) permet désormais de collecter de grandes quantités de données comportementales sur les étudiants, ouvrant la voie à des approches prédictives fondées sur l’analyse de ces traces d’apprentissage.

C’est dans ce contexte que s’inscrit le projet **BrainOps**, dont l’objectif est de concevoir un système capable de prédire le risque de décrochage universitaire à partir de données comportementales issues du jeu de données *xAPI-Edu-Data* (Kaggle). Le projet met en œuvre une architecture complète de type **MLOps**, structurée en plusieurs étapes : l’ingestion des données brutes (**dlt**, **DuckDB**), leur transformation et le contrôle de leur qualité (**dbt**), l’analyse exploratoire et la modélisation par apprentissage automatique, le suivi des expérimentations et le versionnement des modèles (**MLflow**), le déploiement du modèle sous forme de service accessible (**FastAPI**, **Docker**), ainsi que l’automatisation et l’orchestration de l’ensemble du pipeline (**GitHub Actions**, **Dagster**).

Le projet a été mené par une équipe de huit membres selon la méthodologie **Agile Scrum**, avec une gestion opérationnelle du travail via l’outil **Jira** (Product Backlog, User Stories, sprints, revues). Cette organisation a permis de structurer le développement de manière itérative, en assurant à chaque étape une validation métier des livrables techniques produits.

**Mots-clés :** décrochage universitaire, Machine Learning, MLOps, xAPI-Edu-Data, Dagster, Scrum, Jira.

# Table des matières

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Remerciements</b>                                     | <b>1</b>  |
| <b>Résumé</b>                                            | <b>2</b>  |
| <b>Introduction générale</b>                             | <b>10</b> |
| <b>1 Contexte et Cadrage du Projet</b>                   | <b>11</b> |
| 1.1 Introduction . . . . .                               | 11        |
| 1.2 Contexte . . . . .                                   | 11        |
| 1.3 Problématique . . . . .                              | 12        |
| 1.4 Objectifs du projet . . . . .                        | 12        |
| 1.5 Périmètre du projet . . . . .                        | 13        |
| 1.6 Utilisateurs cibles . . . . .                        | 13        |
| 1.6.1 Responsables pédagogiques . . . . .                | 13        |
| 1.6.2 Conseillers et services d'accompagnement . . . . . | 14        |
| 1.6.3 Administration universitaire . . . . .             | 14        |
| 1.6.4 Équipe technique . . . . .                         | 14        |
| 1.7 Valeur métier du projet . . . . .                    | 14        |
| 1.7.1 Détection précoce . . . . .                        | 14        |
| 1.7.2 Aide à la décision . . . . .                       | 15        |
| 1.7.3 Amélioration du suivi des étudiants . . . . .      | 15        |
| 1.7.4 Automatisation . . . . .                           | 15        |
| 1.7.5 Fiabilité des données . . . . .                    | 15        |
| 1.7.6 Traçabilité des modèles . . . . .                  | 15        |
| 1.7.7 Industrialisation . . . . .                        | 15        |
| 1.8 Stratégie des données . . . . .                      | 15        |
| 1.8.1 Source des données . . . . .                       | 16        |
| 1.8.2 Ingestion . . . . .                                | 16        |
| 1.8.3 Stockage . . . . .                                 | 16        |
| 1.8.4 Qualité des données . . . . .                      | 16        |
| 1.8.5 Transformation . . . . .                           | 16        |
| 1.8.6 Préparation pour le Machine Learning . . . . .     | 17        |
| 1.8.7 Gouvernance et traçabilité . . . . .               | 17        |
| 1.9 Architecture fonctionnelle globale . . . . .         | 17        |
| Conclusion du chapitre . . . . .                         | 18        |
| <b>2 Gestion Agile du Projet</b>                         | <b>20</b> |
| 2.1 Organisation de l'équipe . . . . .                   | 20        |
| 2.2 Répartition des rôles . . . . .                      | 20        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 2.3      | Structure du Backlog et des Epics . . . . .                           | 21        |
| 2.3.1    | Organisation des Epics . . . . .                                      | 21        |
| 2.3.2    | Product Backlog . . . . .                                             | 22        |
| 2.3.3    | User Stories principales et critères d'acceptation . . . . .          | 23        |
| 2.4      | Planification des Sprints . . . . .                                   | 25        |
| 2.4.1    | Sprint 1 : DataOps & Qualité . . . . .                                | 26        |
| 2.4.2    | Sprint 2 : ML & MLOps . . . . .                                       | 27        |
| 2.4.3    | Sprint 3 : Déploiement & Industrialisation . . . . .                  | 29        |
| 2.5      | Gestion des anomalies et correction des bugs . . . . .                | 31        |
| 2.6      | Organisation Git et GitHub . . . . .                                  | 31        |
| 2.6.1    | Workflow Git . . . . .                                                | 32        |
| 2.7      | Analyse de la progression du projet . . . . .                         | 32        |
|          | Conclusion du chapitre . . . . .                                      | 33        |
| <b>3</b> | <b>Ingénierie des Données et DataOps</b>                              | <b>34</b> |
| 3.1      | Introduction . . . . .                                                | 34        |
| 3.2      | Description de la source de données . . . . .                         | 34        |
| 3.3      | Standardisation et reproductibilité de l'environnement . . . . .      | 35        |
| 3.4      | Développement du pipeline d'ingestion avec dlt . . . . .              | 36        |
| 3.5      | Persistance et stockage des données (DuckDB) . . . . .                | 36        |
| 3.6      | Validation et contrôle de l'intégrité des données . . . . .           | 37        |
| 3.7      | La couche DataOps avec dbt . . . . .                                  | 37        |
| 3.7.1    | Objectif . . . . .                                                    | 37        |
| 3.7.2    | Conception de la chaîne de transformation . . . . .                   | 38        |
| 3.7.3    | Préparation des variables . . . . .                                   | 38        |
| 3.7.4    | Contrôle de qualité et validation . . . . .                           | 38        |
| 3.7.5    | Organisation des fichiers dbt . . . . .                               | 39        |
| 3.8      | Qualité des données . . . . .                                         | 40        |
| 3.8.1    | Data Contract . . . . .                                               | 40        |
| 3.8.2    | Contrôles techniques — quality_checks.py . . . . .                    | 41        |
| 3.8.3    | Règles métier — check_business_rule.py . . . . .                      | 42        |
| 3.8.4    | Gestion des doublons métier . . . . .                                 | 42        |
| 3.8.5    | Lineage des données . . . . .                                         | 43        |
|          | Conclusion du chapitre . . . . .                                      | 44        |
| <b>4</b> | <b>Modélisation Prédictive et MLOps</b>                               | <b>45</b> |
| 4.1      | Introduction . . . . .                                                | 45        |
| 4.2      | Analyse comparative et stratégie de sélection algorithmique . . . . . | 45        |
| 4.3      | Évaluation holistique de la performance . . . . .                     | 46        |
| 4.3.1    | Le paradoxe de l'Accuracy dans le contexte éducatif . . . . .         | 46        |
| 4.3.2    | La priorité au Rappel (Recall) : la sécurité avant tout . . . . .     | 46        |
| 4.3.3    | Précision et gestion des ressources : éviter la saturation . . . . .  | 47        |
| 4.3.4    | Le modèle comme outil de pilotage stratégique . . . . .               | 47        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 4.4      | Diagnostic de robustesse et analyse des erreurs . . . . .          | 47        |
| 4.5      | Mise en œuvre MLOps : suivi, versionnement et monitoring . . . . . | 48        |
| 4.5.1    | Contexte et objectifs . . . . .                                    | 48        |
| 4.5.2    | Architecture mise en place . . . . .                               | 49        |
| 4.5.3    | Experiment Tracking . . . . .                                      | 49        |
| 4.5.4    | Model Registry — Versionnement . . . . .                           | 50        |
| 4.5.5    | Monitoring du modèle . . . . .                                     | 50        |
| 4.5.6    | Étude de cas : détection d’une anomalie de pipeline . . . . .      | 51        |
| 4.5.7    | Intégration avec l’orchestration automatisée (Dagster) . . . . .   | 52        |
|          | Conclusion du chapitre . . . . .                                   | 53        |
| <b>5</b> | <b>Orchestration de Données avec Dagster</b>                       | <b>54</b> |
| 5.1      | Introduction . . . . .                                             | 54        |
| 5.2      | Présentation de Dagster . . . . .                                  | 55        |
| 5.3      | Architecture de l’orchestration . . . . .                          | 55        |
| 5.4      | Organisation du pipeline sous Dagster . . . . .                    | 56        |
| 5.5      | Orchestration du pipeline Data et Machine Learning . . . . .       | 57        |
| 5.6      | Intégration de MLflow . . . . .                                    | 57        |
| 5.7      | Configuration et automatisation du pipeline . . . . .              | 57        |
| 5.8      | Supervision et suivi des exécutions . . . . .                      | 58        |
|          | Conclusion du chapitre . . . . .                                   | 58        |
| <b>6</b> | <b>Déploiement REST, Conteneurisation et Industrialisation</b>     | <b>60</b> |
| 6.1      | Introduction . . . . .                                             | 60        |
| 6.2      | Architecture de déploiement . . . . .                              | 60        |
| 6.3      | Conteneurisation de l’API avec Docker . . . . .                    | 61        |
| 6.3.1    | Dockerfile . . . . .                                               | 61        |
| 6.3.2    | Fichier de dépendances . . . . .                                   | 62        |
| 6.4      | Orchestration avec Docker Compose . . . . .                        | 62        |
| 6.4.1    | Lancement isolé de l’API . . . . .                                 | 62        |
| 6.4.2    | Orchestration globale . . . . .                                    | 62        |
| 6.5      | Implémentation de l’API . . . . .                                  | 63        |
| 6.5.1    | Point d’entrée . . . . .                                           | 63        |
| 6.5.2    | Schémas de validation . . . . .                                    | 63        |
| 6.5.3    | Endpoints . . . . .                                                | 63        |
| 6.6      | Lancement et test du service . . . . .                             | 64        |
| 6.6.1    | Lancement en local . . . . .                                       | 64        |
| 6.6.2    | Test et validation . . . . .                                       | 64        |
| 6.7      | Choix techniques . . . . .                                         | 65        |
| 6.8      | Intégration Continue (CI) . . . . .                                | 65        |
| 6.9      | Déploiement sur Komodo . . . . .                                   | 66        |
| 6.10     | Monitoring du service . . . . .                                    | 67        |
|          | Conclusion du chapitre . . . . .                                   | 68        |

|                                                |           |
|------------------------------------------------|-----------|
| <b>Conclusion générale et perspectives</b>     | <b>69</b> |
| <b>A Accès au dépôt GitHub du projet</b>       | <b>71</b> |
| <b>B Structure du dépôt GitHub</b>             | <b>72</b> |
| <b>C Extrait du fichier docker-compose.yml</b> | <b>74</b> |
| <b>D Commandes utiles</b>                      | <b>76</b> |

# Table des figures

|     |                                                                                                                                                |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Architecture fonctionnelle globale du pipeline BrainOps . . . . .                                                                              | 18 |
| 2.1 | Extrait du Product Backlog sur Jira et priorisation . . . . .                                                                                  | 22 |
| 2.2 | Sprint 1 sur Jira . . . . .                                                                                                                    | 26 |
| 2.3 | Sprint 2 sur Jira . . . . .                                                                                                                    | 28 |
| 2.4 | Sprint 3 sur Jira . . . . .                                                                                                                    | 29 |
| 2.5 | Gestion d'une anomalie ML (Bug) sous Jira . . . . .                                                                                            | 31 |
| 2.6 | Stratégie de branches adoptée . . . . .                                                                                                        | 31 |
| 2.7 | Analyse de la progression du projet . . . . .                                                                                                  | 32 |
| 3.1 | Source des données sur la plateforme Kaggle . . . . .                                                                                          | 35 |
| 3.2 | Aperçu du jeu de données brut (format CSV) . . . . .                                                                                           | 35 |
| 3.3 | Script d'ingestion automatisé avec dlt et nettoyage des colonnes . . . . .                                                                     | 36 |
| 3.4 | Confirmation du succès de l'ingestion et chargement des données dans DuckDB . . . . .                                                          | 37 |
| 3.5 | Résultat du test de validation et aperçu des données stockées en base . . . . .                                                                | 37 |
| 3.6 | Extrait du schéma YAML de validation et de contrôle de conformité des données . . . . .                                                        | 41 |
| 3.7 | Résultat du contrôle de conformité métier . . . . .                                                                                            | 43 |
| 4.1 | Répartition de la population étudiante par classe de risque académique . . . . .                                                               | 46 |
| 4.2 | Matrice de confusion du modèle Random Forest sur l'ensemble de test . . . . .                                                                  | 47 |
| 4.3 | Analyse de l'engagement comportemental et de l'absentéisme . . . . .                                                                           | 48 |
| 4.4 | Historique des runs MLflow : suivi des expérimentations et versions successives du modèle enregistré dans le Model Registry (v2, v3) . . . . . | 50 |
| 4.5 | Dashboard de monitoring du modèle : évolution de l'accuracy / F1-score et de la dérive maximale détectée dans le temps . . . . .               | 52 |
| 5.1 | Architecture globale du pipeline de données orchestré par Dagster . . . . .                                                                    | 56 |
| 5.2 | Graphe des assets Dagster du projet . . . . .                                                                                                  | 56 |
| 5.3 | État d'exécution et de matérialisation des assets Dagster . . . . .                                                                            | 58 |
| 6.1 | Architecture générale du déploiement (API, MLflow, Dagster) . . . . .                                                                          | 61 |
| 6.2 | Dockerfile de l'API . . . . .                                                                                                                  | 62 |
| 6.3 | Extrait du code de l'endpoint /predict (api/routes.py) . . . . .                                                                               | 63 |
| 6.4 | Documentation interactive Swagger de l'API . . . . .                                                                                           | 64 |
| 6.5 | Conteneurs Docker en cours d'exécution . . . . .                                                                                               | 65 |
| 6.6 | Réponse de l'endpoint /predict dans Swagger . . . . .                                                                                          | 65 |
| 6.7 | Exécution réussie du pipeline d'intégration continue (GitHub Actions) . . . . .                                                                | 66 |

|      |                                                                                                 |    |
|------|-------------------------------------------------------------------------------------------------|----|
| 6.8  | État du déploiement sur la plateforme Cloud : services opérationnels (statut RUNNING) . . . . . | 67 |
| 6.9  | Vérification du bon fonctionnement du service depuis un réseau externe . .                      | 67 |
| 6.10 | Évolution du temps de réponse et de la disponibilité du service . . . . .                       | 68 |

# Liste des tableaux

|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 2.1 | Répartition des rôles au sein de l'équipe . . . . .                | 21 |
| 2.2 | Structuration du backlog par Epics et objectifs associés . . . . . | 22 |
| 2.3 | Synthèse du découpage des Sprints . . . . .                        | 26 |
| 3.1 | Plan des tests de validation et de qualité . . . . .               | 39 |
| 3.2 | Règle métier du niveau d'absence des étudiants . . . . .           | 42 |
| 3.3 | Bilan de l'exécution et de la qualité des données . . . . .        | 43 |
| 4.1 | Métriques enregistrées . . . . .                                   | 49 |
| 4.2 | L'anomalie de pipeline . . . . .                                   | 51 |
| 4.3 | Difficultés rencontrées et solutions . . . . .                     | 53 |
| 6.1 | Bilans temporels et statuts du pipeline . . . . .                  | 66 |
| 6.2 | Synthèse des performances système . . . . .                        | 67 |

# Introduction générale

L'enseignement supérieur traverse actuellement une phase de transformation numérique sans précédent, caractérisée par la généralisation des environnements numériques d'apprentissage (*Learning Management Systems*). Ces plateformes centralisent une quantité considérable de données comportementales et interactionnelles reflétant l'activité quotidienne des apprenants.

Dans le même temps, les universités et les établissements d'enseignement supérieur sont confrontés à une problématique persistante : le décrochage et l'échec universitaire, qui compromettent l'insertion professionnelle des étudiants et pèsent lourdement sur l'efficacité des cursus.

Traditionnellement, la détection des difficultés académiques intervient à l'issue des examens de fin de semestre ou d'année, un stade où les mesures correctives s'avèrent souvent tardives et inefficaces. C'est pour répondre à cette lacune que l'analyse des données éducatives (*Educational Data Mining*) [10] et l'intelligence artificielle trouvent toute leur pertinence.

C'est dans ce cadre que s'inscrit le projet **BrainOps**. Développé dans le cadre de notre formation en Master, ce projet a pour ambition de concevoir un système de classification automatisée des risques académiques fondé sur l'exploitation du dataset *xAPI-Edu-Data* [2, 1].

# Chapitre 1

## Contexte et Cadrage du Projet

### 1.1 Introduction

L’enseignement supérieur contemporain fait face à des mutations profondes, accélérées par la massification des effectifs étudiants et la transition numérique des campus. Dans ce contexte, la réussite académique et la persévérance des étudiants constituent des enjeux majeurs tant pour les individus que pour les institutions. Le décrochage universitaire, qu’il se solde par un abandon pur et simple ou par un échec répété, représente une problématique complexe aux répercussions multiples : humaines, sociales et économiques. Face à ce défi persistant, l’émergence des technologies d’analyse des données éducatives (*Educational Data Mining*) et du Machine Learning offre de nouvelles perspectives pour anticiper ces ruptures de parcours.

C’est précisément dans cette dynamique que s’inscrit le projet **BrainOps**. Conçu comme un système intelligent de classification et d’aide à la décision, BrainOps vise à modéliser et prédire les risques de décrochage et les niveaux de performance des étudiants. En exploitant un ensemble de données comportementales, démographiques et académiques issues des plateformes d’apprentissage en ligne (notamment le dataset de référence *xAPI-Edu-Data*), ce projet ambitionne de fournir aux structures éducatives des outils analytiques rigoureux pour identifier précocement les profils vulnérables et optimiser l’accompagnement pédagogique.

### 1.2 Contexte

L’abandon universitaire, également appelé décrochage académique, constitue un phénomène auquel de nombreux établissements d’enseignement supérieur sont confrontés. Il se traduit par l’interruption du parcours d’un étudiant avant l’obtention du diplôme visé, que ce soit par abandon volontaire, échec académique répété ou réorientation non anticipée. Ce phénomène représente un enjeu à la fois humain, pédagogique et institutionnel :

- **Sur le plan humain** : il peut avoir des conséquences importantes sur la trajectoire professionnelle et personnelle de l’étudiant concerné ;
- **Sur le plan pédagogique** : il interroge la capacité des dispositifs d’accompagnement à détecter et à prévenir les situations à risque ;
- **Sur le plan institutionnel** : il affecte les indicateurs de réussite des établissements ainsi que l’allocation de leurs ressources.

Avec la généralisation des environnements numériques d'apprentissage, une quantité croissante de données relatives au comportement et à l'engagement des étudiants est désormais collectée [10].

## 1.3 Problématique

Le décrochage universitaire est un phénomène multifactoriel, difficile à anticiper à partir des seuls indicateurs académiques traditionnels (notes, présence aux examens), souvent disponibles trop tardivement pour permettre une intervention efficace. La problématique centrale de ce projet peut ainsi être formulée de la manière suivante :

### Problématique centrale

*« Comment détecter, de manière automatisée, fiable et suffisamment précoce, les étudiants présentant un risque élevé de décrochage universitaire, à partir de l'analyse de leurs données comportementales d'apprentissage ? »*

Cette problématique soulève plusieurs sous-questions qui ont guidé la conception du projet :

- Quelles variables comportementales sont réellement prédictives du risque de décrochage ?
- Comment structurer un pipeline de données fiable, reproductible et automatisé, depuis la donnée brute jusqu'à la prédiction ?
- Quel modèle de Machine Learning offre le meilleur compromis entre performance et interprétabilité pour ce cas d'usage ?
- Comment rendre cette prédiction exploitable de manière opérationnelle par les acteurs pédagogiques concernés ?

## 1.4 Objectifs du projet

L'objectif général du projet BrainOps est de concevoir, développer et déployer un système automatisé de prédiction du risque de décrochage universitaire, fondé sur l'analyse de données comportementales d'apprentissage.

Cet objectif général se décline en plusieurs objectifs spécifiques :

1. Mettre en place une chaîne d'ingestion fiable des données brutes issues du jeu de données *xAPI-Edu-Data* ;
2. Transformer et fiabiliser ces données afin de garantir leur qualité pour les usages analytiques et prédictifs ;
3. Explorer les données et développer un modèle de classification du risque de décrochage ;
4. Assurer la traçabilité et la reproductibilité des expérimentations réalisées (suivi des modèles et de leurs performances) ;

5. Déployer le modèle retenu sous la forme d'un service accessible et exploitable ;
6. Automatiser et orchestrer l'ensemble du pipeline afin de limiter les interventions manuelles ;
7. Structurer le travail d'équipe selon une démarche Agile rigoureuse, avec un outillage de gestion de projet adapté (Jira).

## 1.5 Périmètre du projet

Le projet BrainOps couvre la conception et la mise en œuvre d'une solution complète de Data Science, Machine Learning et MLOps dédiée à la prédiction du risque de décrochage universitaire. Le périmètre comprend :

- L'ingestion du dataset *xAPI-Edu-Data* à l'aide de `dlt` ;
- Son stockage dans `DuckDB` ;
- Sa transformation et sa validation avec `dbt` ;
- L'analyse exploratoire, le Feature Engineering et l'entraînement de modèles de classification ;
- `MLflow` pour le suivi des expériences et la gestion des versions des modèles ;
- `Dagster` pour l'orchestration du pipeline ;
- `FastAPI` et `Docker` pour l'exposition et la conteneurisation du modèle ;
- Une démarche CI/CD avec `GitHub Actions` et un déploiement cloud.

BrainOps constitue ainsi un outil d'aide à la décision destiné à favoriser la détection précoce des étudiants potentiellement à risque, sans se substituer à la décision des responsables pédagogiques, conformément aux principes du MLOps [12].

## 1.6 Utilisateurs cibles

BrainOps s'adresse principalement aux acteurs impliqués dans le suivi et l'accompagnement des étudiants. Quatre catégories d'utilisateurs peuvent être distinguées :

### 1.6.1 Responsables pédagogiques

Les responsables pédagogiques constituent l'un des principaux utilisateurs cibles du système. Ils peuvent exploiter les résultats produits par BrainOps afin d'identifier les étudiants nécessitant éventuellement une attention particulière, et ainsi mieux orienter les actions d'accompagnement mises en place au sein de l'établissement.

**Besoin principal** : disposer d'informations permettant d'identifier rapidement les profils présentant un risque académique.

## 1.6.2 Conseillers et services d'accompagnement

Les conseillers pédagogiques peuvent utiliser les résultats du système comme une information complémentaire dans le cadre de l'accompagnement individuel des étudiants, en appui de leur propre analyse de la situation de chacun.

**Besoin principal** : disposer d'indicateurs permettant de cibler les actions d'accompagnement.

## 1.6.3 Administration universitaire

L'administration peut exploiter les résultats agrégés produits par le système afin de mieux comprendre les tendances liées aux difficultés académiques à l'échelle d'une promotion ou d'un établissement.

**Besoin principal** : disposer d'indicateurs permettant d'améliorer le pilotage et la prévention du décrochage.

## 1.6.4 Équipe technique

L'équipe Data/IT constitue également un utilisateur important de la plateforme, dans la mesure où elle est responsable du bon fonctionnement du système dans la durée. Son rôle recouvre notamment :

- l'exploitation du pipeline de données ;
- la maintenance de la plateforme ;
- la surveillance des modèles en production ;
- la gestion des données ;
- le déploiement des nouvelles versions ;
- l'évolution générale de la solution.

## 1.7 Valeur métier du projet

BrainOps apporte une valeur métier à plusieurs niveaux, qui dépassent la seule performance technique des modèles de classification et concernent l'ensemble du cycle de vie de la donnée jusqu'à son exploitation opérationnelle.

### 1.7.1 Détection précoce

La solution permet d'exploiter les données disponibles afin d'identifier plus rapidement les profils d'étudiants nécessitant éventuellement une attention particulière, avant que leur situation ne se dégrade davantage.

### **1.7.2 Aide à la décision**

Le système fournit des prédictions pouvant compléter l'analyse réalisée par les responsables pédagogiques. Il ne remplace donc pas l'expertise humaine, mais constitue un support supplémentaire à la prise de décision.

### **1.7.3 Amélioration du suivi des étudiants**

L'exploitation systématique des données peut contribuer à la mise en place d'un suivi plus ciblé et mieux adapté aux besoins réellement identifiés pour chaque étudiant.

### **1.7.4 Automatisation**

L'utilisation de `dlt`, `dbt` et `Dagster` permet d'automatiser plusieurs opérations qui pourraient autrement être réalisées manuellement, réduisant ainsi le risque d'erreur et le temps consacré aux tâches répétitives de préparation des données.

### **1.7.5 Fiabilité des données**

Les contrôles de qualité et les transformations mises en œuvre permettent de réduire les risques liés à l'utilisation de données incorrectes ou incohérentes dans le processus de modélisation.

### **1.7.6 Traçabilité des modèles**

`MLflow` permet de conserver l'historique des expériences et des versions de modèles, facilitant ainsi leur comparaison et leur reproductibilité tout au long du projet.

### **1.7.7 Industrialisation**

L'intégration de `Docker`, `FastAPI` et d'une chaîne d'intégration et de déploiement continu (CI/CD) permet de rapprocher le projet d'une architecture exploitable dans un environnement réel, au-delà du seul cadre expérimental.

## **1.8 Stratégie des données**

La Data Strategy de BrainOps définit la manière dont les données sont collectées, stockées, transformées, contrôlées et exploitées tout au long du projet. Son objectif est de garantir que les données utilisées par les modèles sont accessibles, fiables, traçables et adaptées aux besoins du Machine Learning. Cette stratégie est organisée selon sept étapes.

### 1.8.1 Source des données

La source principale utilisée dans le projet est le dataset *xAPI-Edu-Data*, qui contient des informations relatives au comportement et à la situation académique des étudiants. Avant toute utilisation, les données doivent être analysées afin d'en comprendre la structure, les types, les valeurs, les éventuelles anomalies, ainsi que la variable cible.

### 1.8.2 Ingestion

L'ingestion des données est réalisée avec `dlt`. Cette étape permet de transférer les données depuis leur source vers l'environnement de stockage, tout en veillant à ce que cette opération soit reproductible et automatisable.

### 1.8.3 Stockage

Les données brutes sont stockées dans DuckDB. Une séparation logique entre les données brutes et les données transformées permet de conserver une meilleure traçabilité tout au long du processus. La stratégie de stockage peut ainsi être représentée selon la progression suivante :

Source → Raw Data → Clean Data → Analytical Data → ML Dataset

### 1.8.4 Qualité des données

La qualité des données constitue une étape essentielle de la stratégie. Les contrôles mis en œuvre portent notamment sur :

- les valeurs manquantes ;
- les doublons ;
- les types de données ;
- les valeurs incohérentes ;
- les contraintes métier ;
- la complétude des données.

### 1.8.5 Transformation

Les transformations sont réalisées avec `dbt`. Cette couche permet de produire des données structurées et adaptées aux besoins de l'analyse et du Machine Learning. Elle permet également de documenter les transformations appliquées et de faciliter le suivi du Data Lineage.

### 1.8.6 Préparation pour le Machine Learning

Une fois les données transformées, elles sont préparées en vue de leur exploitation par les modèles de Machine Learning. Cette étape comprend notamment :

- la sélection des variables ;
- le traitement des variables catégorielles ;
- le traitement des valeurs manquantes ;
- le Feature Engineering ;
- la séparation des données d’entraînement et de test.

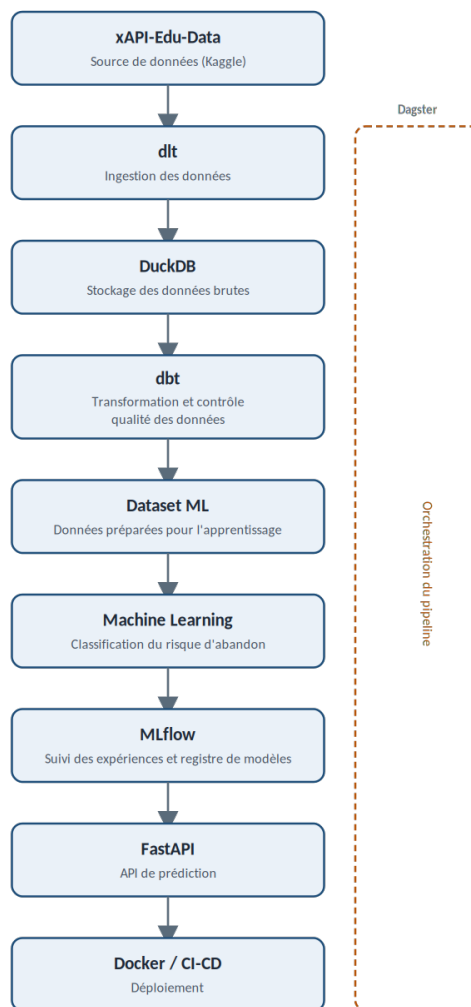
### 1.8.7 Gouvernance et traçabilité

La stratégie Data prévoit également une attention particulière portée à la traçabilité de l’ensemble du processus. Il s’agit notamment de pouvoir répondre, à tout moment, aux questions suivantes : d’où provient une donnée donnée, quelle transformation lui a été appliquée, quelle version du dataset a été utilisée, quel modèle a été entraîné à partir de ces données, et avec quels paramètres et quelles performances. Cette traçabilité est assurée par la combinaison de `dbt`, du Data Lineage, de `Git` et de `MLflow`.

## 1.9 Architecture fonctionnelle globale

La Figure 1.1 présente le fonctionnement global du projet. Les données des étudiants sont d’abord récupérées depuis le jeu de données *xAPI-Edu-Data*, puis stockées dans `DuckDB` à l’aide de `dlt`. Elles sont ensuite nettoyées et transformées avec `dbt` afin d’obtenir un dataset prêt pour l’apprentissage automatique.

Un modèle de classification est ensuite entraîné pour prédire le risque d’abandon universitaire. Le modèle est suivi et versionné avec `MLflow`, puis rendu accessible à travers une API `FastAPI`. Enfin, `Docker`, la CI/CD et `Dagster` permettent respectivement de conteneuriser, automatiser le déploiement et orchestrer l’ensemble du pipeline.



**Figure 1.1 :** Architecture fonctionnelle globale du pipeline BrainOps

Cette architecture met en évidence deux dimensions complémentaires du projet : d’une part, un pipeline de données linéaire garantissant la qualité et la traçabilité de l’information depuis sa source jusqu’au modèle ; d’autre part, une couche d’orchestration transverse, assurée par **Dagster**, qui coordonne l’ensemble des tâches et permet d’automatiser leur exécution de bout en bout.

## Conclusion du chapitre

Ce premier chapitre a permis de poser le cadre général du projet BrainOps : contexte, problématique, objectifs, périmètre, utilisateurs cibles, valeur métier et stratégie des données. L’architecture fonctionnelle globale a été présentée, mettant en évidence la sépa-

ration entre le pipeline de données et la couche d'orchestration. Le chapitre suivant est consacré à la gestion Agile du projet et à l'organisation de l'équipe.

# Chapitre 2

## Gestion Agile du Projet

### 2.1 Organisation de l'équipe

Le projet « Prédiction de l'abandon universitaire » a été réalisé selon une approche Agile basée sur **Scrum** [11]. Cette organisation nous a permis de répartir les responsabilités entre les membres de l'équipe, de suivre l'avancement du projet et d'intégrer progressivement les différentes composantes du pipeline DataOps et MLOps. L'utilisation de **Jira** a permis de gérer le backlog, les User Stories et les sprints, tandis que **GitHub** a été utilisé pour le versionnement du code source et la collaboration entre les membres de l'équipe.

### 2.2 Répartition des rôles

L'équipe est composée de huit membres. Les responsabilités ont été réparties selon les différentes étapes du cycle de vie DataOps, Machine Learning, MLOps et DevOps.

**Table 2.1** : Répartition des rôles au sein de l'équipe

| Membre          | Rôle principal                                                           | Responsabilités                                                                                                                                                                                  |
|-----------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hajar           | Product Owner<br>+ Orchestration Lead                                    | Définition de la vision, priorisation du besoin, validation fonctionnelle et orchestration du pipeline avec Dagster                                                                              |
| Fatima<br>Zahra | Scrum Master<br>+ Git Manager<br>+ Support ML<br>+ Data Quality Engineer | Gestion Jira, animation des cérémonies, gestion des branches GitHub, Pull Requests, README, support documentation ML, Data Contracts, Data Lineage et définition des règles métier et techniques |
| Doaa            | Data Engineer                                                            | Ingestion du dataset xAPI-Edu-Data, préparation des données brutes et intégration avec les outils DataOps                                                                                        |
| Yousra          | Data/Analytics Engineer                                                  | Transformation des données, préparation des données analytiques et contrôle de leur qualité                                                                                                      |
| Hasnaa          | ML Engineer                                                              | Prétraitement, analyse exploratoire, Feature Engineering, entraînement et évaluation des modèles                                                                                                 |
| Wijdane         | MLOps Engineer                                                           | MLflow, Experiment Tracking, Model Registry, versionnement et suivi du modèle                                                                                                                    |
| Hiba            | Deployment Engineer                                                      | Développement de l'API FastAPI, intégration du modèle et conteneurisation avec Docker                                                                                                            |
| Soukaina        | DevOps & Cloud Engineer                                                  | CI/CD, GitHub Actions, déploiement Cloud et industrialisation du service                                                                                                                         |

## 2.3 Structure du Backlog et des Epics

### 2.3.1 Organisation des Epics

Afin de structurer les exigences du projet, les besoins ont été découpés en **Epics** (grands axes stratégiques) puis déclinés en **User Stories** (US). Pour conserver une lecture fluide du rapport, seules les User Stories clés représentant le cœur fonctionnel et MLOps du système sont détaillées ci-dessous avec leurs critères d'acceptation.

**Table 2.2 :** Structuration du backlog par Epics et objectifs associés

| Epic                                 | Objectif                                                                                               |
|--------------------------------------|--------------------------------------------------------------------------------------------------------|
| Gestion Agile                        | Organiser le projet, gérer Jira, les Sprints, la documentation et la collaboration GitHub              |
| Ingestion des données                | Collecter et intégrer le dataset xAPI-Edu-Data dans le pipeline                                        |
| Transformation & Qualité des données | Nettoyer, transformer et contrôler la qualité des données des données                                  |
| Orchestration de pipeline            | Automatiser l'exécution des différentes étapes du pipeline avec Dagster                                |
| Modélisation ML                      | Préparer les données, entraîner, comparer et évaluer les modèles de classification                     |
| MLOps & Tracking                     | Assurer le suivi des expériences, le versionnement et la traçabilité des modèles avec MLflow           |
| Déploiement & API                    | Exposer le modèle via une API REST et préparer son exécution dans un environnement conteneurisé        |
| CI/CD & Cloud                        | Automatiser les tests, l'intégration, le déploiement et la mise à disposition du service dans le Cloud |

Cette organisation permet de distinguer clairement les différentes responsabilités tout en conservant une cohérence avec l'architecture globale du projet.

### 2.3.2 Product Backlog

Le Product Backlog regroupe les User Stories, les tâches techniques, les sous-tâches et les bugs nécessaires à la réalisation du système. Les tickets sont organisés selon les huit Epics définis précédemment.

|                          |                                                                                                                                                         |                                                                                                        |                                                                                                        |          |                  |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------|------------------|
| <input type="checkbox"/> |  <b>SCRUM-113</b> Baisse de performances à cause du scaler           |  hasnaa echchatbi   |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-112</b> Superviser la synchronisation des branches a...    |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-111</b> Contrôler et valider les Pull Requests avant fu... |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Revue en cours ▼ |
| <input type="checkbox"/> |  <b>SCRUM-110</b> Rédiger le guide Git/GitHub dans le README         |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-109</b> Configurer les règles de Pull Request (fusion ...  |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-108</b> Créer les branches feature/* pour chaque m...      |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-107</b> Créer les branches principales (main et devel...   |  Fatima zahra Ablou |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-105</b> DuckDB                                             |  Doaa Romadi        |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |
| <input type="checkbox"/> |  <b>SCRUM-104</b> stockage initial                                   |  Doaa Romadi        |  Fatima zahra Ab... | = Medium | Terminé(e) ▼     |

**Figure 2.1 :** Extrait du Product Backlog sur Jira et priorisation

### 2.3.3 User Stories principales et critères d'acceptation

#### SCRUM-21 — Ingestion automatique

**User Story :** En tant que Data Engineer, je veux automatiser l'ingestion du dataset afin de rendre les données disponibles pour les étapes suivantes du pipeline.

**Critères d'acceptation :**

- Le dataset est correctement chargé.
- Les colonnes sont standardisées.
- Les données sont stockées dans la source RAW.
- L'exécution du processus se termine sans erreur.
- Le nombre de lignes ingérées est vérifiable.

**Résultat :** Le dataset a été ingéré avec succès et les données sont disponibles dans la source RAW pour les étapes de transformation et de contrôle qualité.

#### SCRUM-27 — Contrôle de la qualité des données

**User Story :** En tant que Data Quality Engineer, je veux contrôler la qualité des données afin de garantir que les données utilisées dans le pipeline Machine Learning sont fiables et conformes aux règles définies.

**Critères d'acceptation :**

- Les colonnes obligatoires sont présentes.
- Les valeurs nulles sont contrôlées conformément au Data Contract.
- Les doublons sont détectés.
- L'unicité des identifiants `_dlt_id` est vérifiée.
- Les doublons métier sont détectés.
- Les règles métier sont vérifiées.
- Un statut global de qualité est généré.

**Résultat :** La Story est considérée comme acceptée lorsque les contrôles définis sont exécutés correctement et que le résultat de qualité est généré.

#### SCRUM-27 — Préparation des données ML

**User Story :** En tant que ML Engineer, je veux préparer les données pour l'entraînement du modèle.

**Critères d'acceptation :**

- Les données validées sont récupérées depuis la source préparée.
- Les variables catégorielles sont correctement traitées.
- Les variables numériques sont correctement préparées.
- Les données sont séparées en jeux Train, Validation et Test.

- Le prétraitement est appliqué de manière cohérente aux différents jeux de données.
- Le scaler est ajusté (**fit**) uniquement sur le jeu d'entraînement.
- Le scaler est ensuite appliqué (**transform**) aux jeux de validation et de test.
- Les données préparées sont sauvegardées dans `data/processed/`.

**Résultat :** Les données ont été préparées pour le processus de Machine Learning. Le prétraitement permet de générer les jeux de données nécessaires à l'entraînement, à la validation et au test du modèle, ainsi que de conserver les informations nécessaires à la réutilisation du même prétraitement lors des futures prédictions.

### SCRUM-30 — Entraîner le modèle

**User Story :** En tant que conseiller pédagogique, je veux obtenir une classification du risque de décrochage par étudiant afin de prioriser mes interventions d'accompagnement.

**Critères d'acceptation :**

- Plusieurs algorithmes de classification sont entraînés.
- Les modèles utilisés comprennent Logistic Regression, Random Forest et XGBoost.
- Chaque modèle est entraîné uniquement sur le jeu d'entraînement.
- Les performances des modèles sont mesurées sur le jeu de validation.
- Les résultats des modèles sont comparables à l'aide des mêmes métriques.
- Le meilleur modèle est sélectionné sur la base des performances de validation.
- Le modèle sélectionné est sauvegardé dans `ml/models/`.
- Le modèle entraîné peut être rechargé pour effectuer des prédictions.

**Résultat :** Plusieurs modèles de classification ont été entraînés et comparés : Logistic Regression, Random Forest et XGBoost. Le Random Forest a obtenu la meilleure Accuracy sur le jeu de validation avec **81,94 %**, devant XGBoost avec 80,56 % et Logistic Regression avec 69,44 %. Le Random Forest a donc été retenu comme meilleur modèle candidat. L'évaluation sur le jeu de test a ensuite donné une Accuracy de **76 %**.

### SCRUM-39 — Développer une API REST

**User Story :** En tant qu'utilisateur, je veux accéder au modèle de prédiction à travers une API REST afin de pouvoir obtenir une prédiction de risque de décrochage universitaire.

**Critères d'acceptation :**

- Une API REST est développée avec FastAPI.
- L'API peut démarrer sans erreur.
- Un endpoint de prédiction est disponible.
- L'API accepte les caractéristiques nécessaires à une prédiction.
- Les données reçues sont validées avant la prédiction.

- Le modèle et le scaler sont correctement chargés.
- L'API retourne une prédiction exploitable.
- Les erreurs de données entrantes sont correctement gérées.
- L'API est accessible depuis l'extérieur du conteneur.
- La documentation interactive de l'API est accessible via Swagger.

**Résultat :** L'API REST développée avec FastAPI permet d'envoyer les caractéristiques d'un étudiant et d'obtenir en réponse la prédiction de son niveau de risque de décrochage. L'API assure également la validation des données reçues et utilise les artefacts du modèle entraîné pour effectuer la prédiction. L'endpoint de prédiction a été testé avec succès à partir de plusieurs exemples et retourne une réponse valide.

## SCRUM-44 — Déploiement automatisé

**User Story :** En tant que DevOps Engineer, je veux automatiser le déploiement de l'application afin de rendre la mise en production reproductible.

**Critères d'acceptation :**

- Le déploiement de l'application peut être déclenché automatiquement.
- La version à déployer provient du dépôt Git configuré.
- Le déploiement utilise la configuration Docker Compose prévue pour le projet.
- Les images nécessaires sont construites ou récupérées correctement.
- Les services sont démarrés après le déploiement.
- L'API est accessible après le déploiement.
- Une vérification permet de confirmer que le service fonctionne.
- Le déploiement d'une nouvelle version ne nécessite pas de modification manuelle du code sur le serveur.
- Le Stack du projet est correctement configuré dans Komodo.
- Le dépôt Git utilisé par le Stack pointe vers la branche develop.
- Le `docker-compose.yml` utilisé est celui du projet.
- Le déploiement est limité aux ressources du groupe.

**Résultat :** Le déploiement est effectué à partir du Stack Komodo associé au dépôt Git du projet. La configuration utilise la branche main et le fichier `docker-compose.yml` situé à la racine du dépôt.

## 2.4 Planification des Sprints

Le projet a été organisé en trois Sprints principaux, chacun correspondant à une étape importante du cycle de développement.

**Table 2.3** : Synthèse du découpage des Sprints

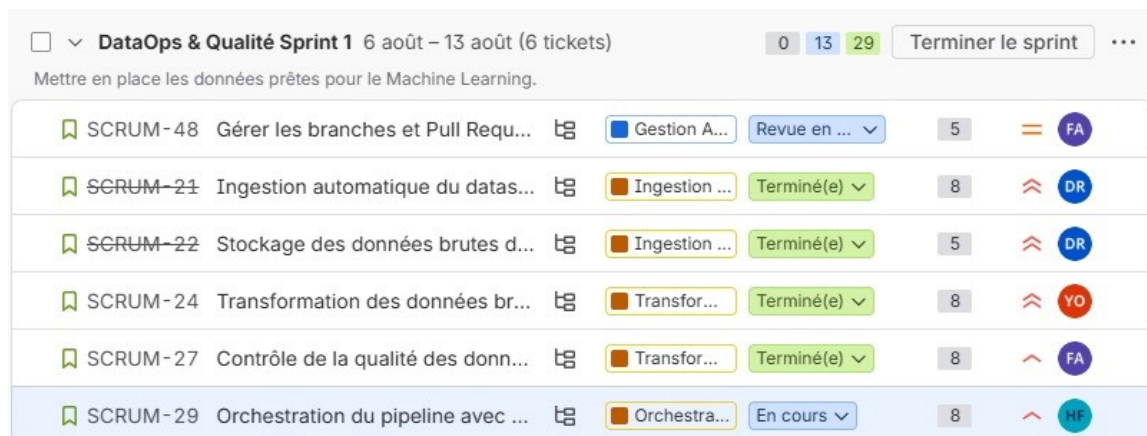
| Sprint   | Nom                             | Objectif principal                              |
|----------|---------------------------------|-------------------------------------------------|
| Sprint 1 | DataOps & Qualité               | Construire et fiabiliser le pipeline de données |
| Sprint 2 | ML & MLOps                      | Construire, évaluer et versionner le modèle     |
| Sprint 3 | Déploiement & Industrialisation | Exposer, conteneuriser et déployer la solution  |

### 2.4.1 Sprint 1 : DataOps & Qualité

#### Sprint Planning

Le premier Sprint est consacré à la préparation d'une base de données fiable pour les étapes de Machine Learning. Les travaux portent principalement sur :

- l'ingestion du dataset ;
- le stockage des données ;
- la transformation ;
- les contrôles de qualité ;
- la préparation du pipeline ;
- l'orchestration avec Dagster.

**Figure 2.2** : Sprint 1 sur Jira

#### Sprint Review

À la fin du Sprint, l'équipe vérifie :

- la disponibilité des données ;
- le bon fonctionnement de l'ingestion ;
- la réussite des transformations ;
- le passage des contrôles de qualité ;
- l'exécution correcte du pipeline orchestré.

## Sprint Retrospective

### Points positifs :

- Répartition claire des responsabilités ;
- utilisation de Jira pour suivre les travaux ;
- utilisation de GitHub pour collaborer ;
- séparation des étapes DataOps et Machine Learning.

### Difficultés rencontrées :

- Coordination entre les différentes étapes du pipeline ;
- synchronisation du travail entre plusieurs branches ;
- dépendance entre la préparation des données et le développement ML.

### Actions d'amélioration :

- Décomposer les grandes User Stories en sous-tâches ;
- effectuer des Pull Requests régulières ;
- documenter les interfaces entre les différentes parties du pipeline.

## 2.4.2 Sprint 2 : ML & MLOps

### Sprint Planning

Le deuxième Sprint constitue le cœur du projet de Machine Learning. L'objectif est de construire un modèle permettant de classer les risques de décrochage universitaire à partir des caractéristiques comportementales et académiques des étudiants. Les travaux prévus sont :

1. Préparer les données ;
2. réaliser l'analyse exploratoire ;
3. effectuer le Feature Engineering ;
4. séparer les données en Train, Validation et Test ;
5. entraîner plusieurs modèles ;
6. comparer leurs performances ;
7. sélectionner le meilleur modèle ;
8. réaliser l'évaluation finale ;
9. générer la matrice de confusion ;
10. versionner les artefacts du modèle ;
11. intégrer le suivi des expériences avec MLflow.

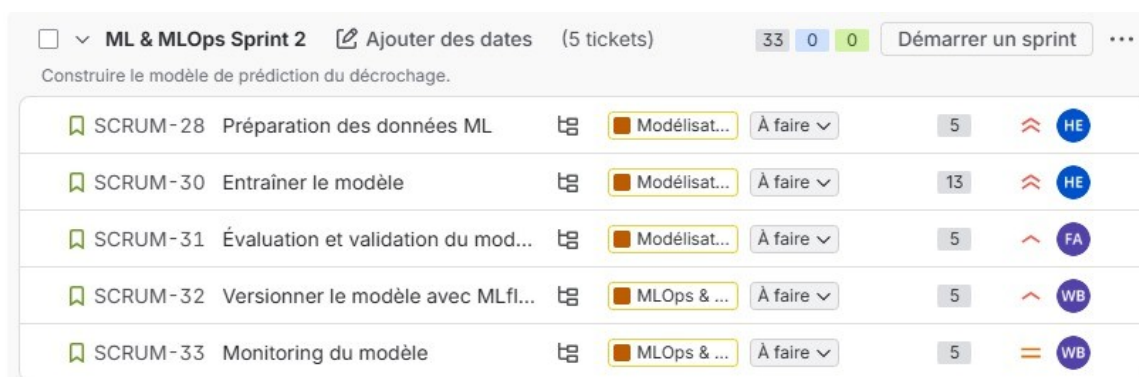


Figure 2.3 : Sprint 2 sur Jira

## Sprint Review

Les données ont été préparées pour l'entraînement et séparées en jeux Train, Validation et Test. Plusieurs algorithmes de classification ont ensuite été comparés, notamment la Logistic Regression, le Random Forest et XGBoost.

Les résultats obtenus sur le jeu de validation montrent que le Random Forest présente les meilleures performances avec une Accuracy de **81,94 %**, contre **80,56 %** pour XGBoost et **69,44 %** pour la Logistic Regression. Le Random Forest a donc été retenu comme modèle candidat.

Une évaluation finale sur le jeu de test a ensuite été réalisée. Le modèle sélectionné obtient une Accuracy de **76 %**. La matrice de confusion et les différentes métriques d'évaluation ont également été utilisées afin d'analyser les erreurs de classification entre les trois catégories de risque.

Le Sprint a également préparé l'intégration du modèle dans la partie MLOps, notamment à travers la sauvegarde des artefacts nécessaires à sa reproductibilité et l'intégration avec MLflow.

## Sprint Retrospective

### Points positifs :

- Les données nécessaires à l'entraînement étaient disponibles.
- Plusieurs modèles ont pu être comparés selon les mêmes critères.
- La séparation entre Train, Validation et Test a permis d'évaluer correctement la généralisation du modèle.
- Le Random Forest a obtenu les meilleures performances sur le jeu de validation.
- Les artefacts nécessaires à la réutilisation du modèle ont été sauvegardés.

### Difficultés rencontrées :

- La préparation des données devait rester cohérente entre l'entraînement et les futures prédictions.
- Une attention particulière a été nécessaire concernant l'utilisation du scaler afin d'éviter les fuites de données.

- Les performances sur le jeu de test sont inférieures à celles observées sur la validation.
- L'interprétation du Recall de la classe **High Risk** reste importante, celui-ci étant actuellement de **0,62**.
- L'intégration du modèle avec MLflow nécessite une coordination avec les composants MLOps.

#### Actions d'amélioration :

- Vérifier systématiquement le pipeline de prétraitement avant chaque entraînement.
- Conserver les mêmes artefacts de prétraitement pour les futures prédictions.
- Suivre plusieurs métriques au lieu de se limiter à l'Accuracy.
- Améliorer progressivement le suivi des expériences avec MLflow.
- Préparer l'intégration du modèle avec l'API avant le Sprint de déploiement.

### 2.4.3 Sprint 3 : Déploiement & Industrialisation

#### Sprint Planning

Le troisième Sprint est consacré à l'industrialisation de la solution Machine Learning. Les principaux travaux sont :

- développement de l'API FastAPI ;
- intégration du modèle ;
- création des endpoints ;
- conteneurisation Docker ;
- orchestration des services ;
- mise en place de la CI/CD ;
- déploiement manuel sur Komodo.

L'objectif final est de rendre le système accessible et reproductible dans un environnement proche d'un contexte réel de production.

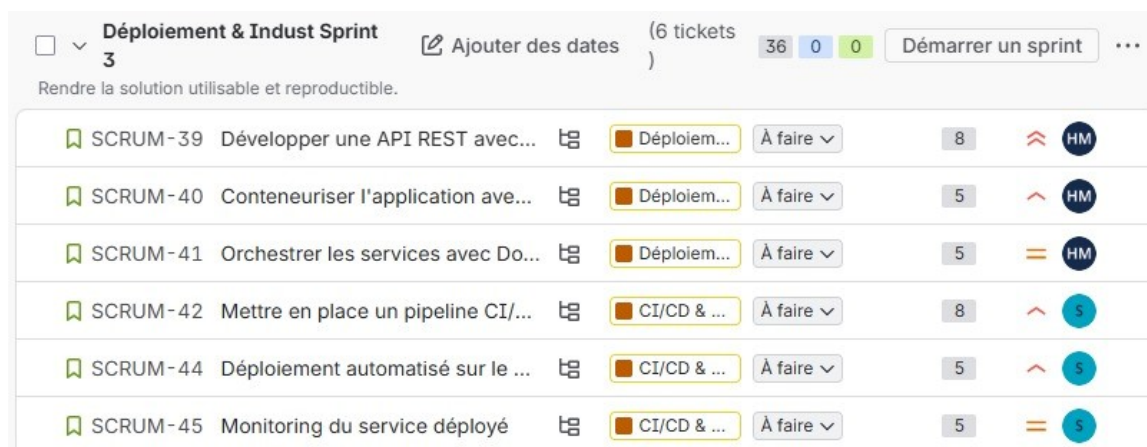


Figure 2.4 : Sprint 3 sur Jira

## Sprint Review

À l'issue du Sprint 3, l'équipe a évalué les fonctionnalités liées à l'industrialisation et au déploiement de la solution. L'objectif était de rendre le modèle accessible à travers une API REST et de préparer son exécution dans un environnement conteneurisé et reproductible.

L'API FastAPI permet d'exposer le modèle de prédiction à travers un endpoint dédié. L'application est ensuite conteneurisée avec Docker afin de garantir un environnement d'exécution reproductible.

Les différents services nécessaires au fonctionnement de la solution sont regroupés à l'aide de Docker Compose. Le déploiement est ensuite préparé sur Komodo.

Enfin, des mécanismes de vérification du fonctionnement du service et d'accès aux logs sont prévus afin de faciliter le suivi de l'application après son déploiement.

## Sprint Retrospective

### Points positifs :

- La séparation entre le développement de l'application et son déploiement facilite l'industrialisation.
- Docker permet de disposer d'un environnement d'exécution reproductible.
- Docker Compose permet de centraliser la configuration des différents services.
- L'utilisation de GitHub Actions permet d'automatiser les vérifications du projet.
- Le déploiement sur l'environnement prévu permet de rapprocher la solution d'un contexte de production.

### Difficultés rencontrées :

- La coordination entre l'API, le modèle et les différents services Docker.
- La gestion des dépendances entre les conteneurs.
- La configuration des ports et des volumes dans l'environnement de déploiement.
- La nécessité de vérifier que les artefacts du modèle sont accessibles par l'API.
- La configuration du déploiement et des mécanismes de monitoring.

### Actions d'amélioration :

- Standardiser la configuration Docker.
- Ajouter des vérifications automatiques avant chaque déploiement.
- Mettre en place un health check de l'API.
- Améliorer la gestion des logs.
- Automatiser davantage le processus de déploiement.
- Documenter la procédure de déploiement et de restauration.

## 2.5 Gestion des anomalies et correction des bugs

Au cours du développement des composants Machine Learning et MLOps, un processus rigoureux de gestion des incidents et des anomalies a été appliqué afin d'assurer la qualité et la fiabilité des modèles. Lorsqu'une anomalie a été identifiée, un ticket Bug a été créé dans Jira afin de documenter le problème, suivre sa résolution et conserver une trace de la correction.

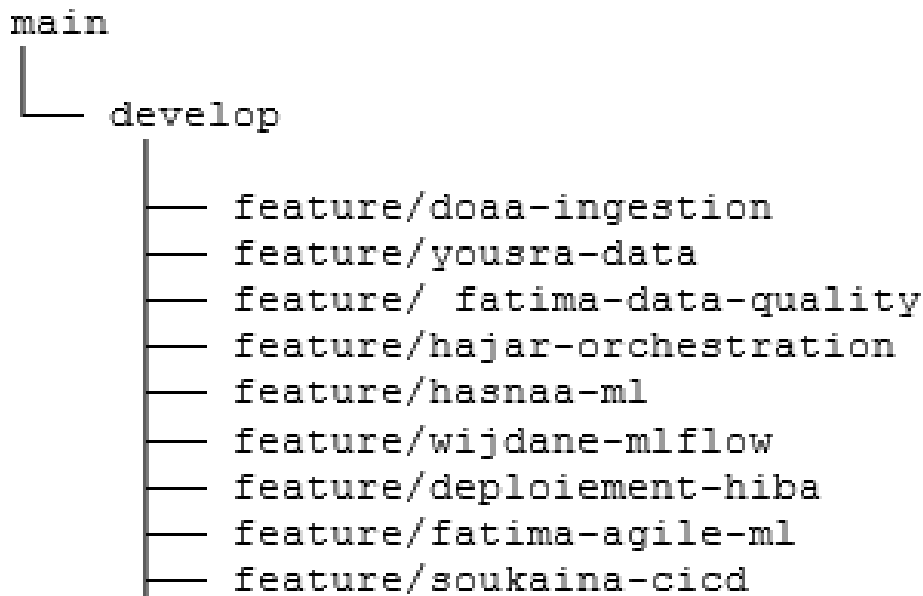


**Figure 2.5 :** Gestion d'une anomalie ML (Bug) sous Jira

- **Description du problème :** Une dégradation des performances du modèle de classification a été observée lors de l'évaluation sur le jeu de test.
- **Action corrective apportée :** Reconfiguration du pipeline de prétraitement pour s'assurer que le scaler est ajusté (fit) exclusivement sur le jeu d'entraînement (Train Set).

## 2.6 Organisation Git et GitHub

Afin d'éviter les conflits et de garantir un historique propre, le projet utilise une stratégie de branches basée sur deux branches principales :



**Figure 2.6 :** Stratégie de branches adoptée

- La branche `main` représente la version stable du projet.

- La branche `develop` constitue la branche principale d'intégration.
- Chaque membre travaille sur une branche dédiée correspondant à sa fonctionnalité : `feature/doaa-ingestion`, `feature/yousra-data`, `feature/fatima-data-quality`, `feature/hajar-orchestration`, `feature/hasnaa-ml`, `feature/wijdane-mlflow`, `feature/deploy`, `feature/fatima-agile-ml`, `feature/soukaina-cicd`.

### 2.6.1 Workflow Git

Le workflow adopté est le suivant :

1. **Mise à jour de develop** : `git checkout develop` puis `git pull origin develop`
2. **Création ou utilisation de sa branche** : `git checkout -b feature/nom-de-la-fonctionnalite`
3. **Développement et tests** : Les modifications sont réalisées uniquement sur la branche personnelle.
4. **Commit** : `git add .` puis `git commit -m "feat: description de la modification"`
5. **Push** : `git push origin feature/nom-de-la-fonctionnalite`
6. **Pull Request** : Une Pull Request est ensuite créée vers `develop`.
7. **Revue de code** : La Pull Request est vérifiée avant sa fusion. Le Git Manager assure le suivi des branches, des Pull Requests et de la cohérence du dépôt.
8. **Fusion vers main** : Lorsque la version intégrée est considérée comme stable : `develop` -> `main`. La branche `main` ne doit pas recevoir de développement direct.

## 2.7 Analyse de la progression du projet

L'analyse de l'avancement montre que la majorité des tâches prévues ont été réalisées ou sont en cours de finalisation. Les tâches relatives au DataOps et à la qualité des données ont été finalisées avant l'intégration des composants Machine Learning. Cette progression reflète les dépendances techniques.

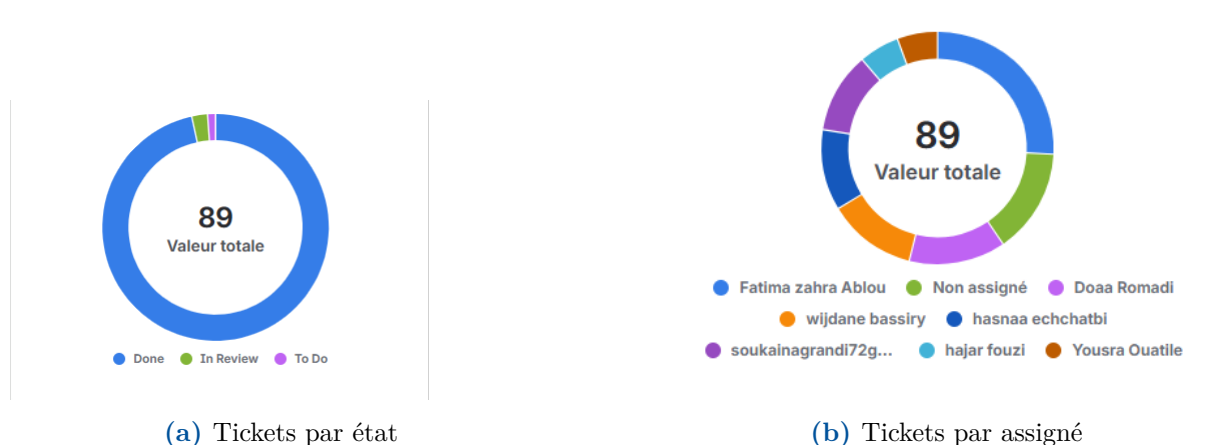


Figure 2.7 : Analyse de la progression du projet

## Conclusion du chapitre

L'approche Agile Scrum permet de structurer le développement du système de prédiction de l'abandon universitaire autour de trois étapes majeures : DataOps & Qualité, ML & MLOps et Déploiement & Industrialisation. La décomposition du projet en huit Epics permet de représenter les différentes dimensions du système : gestion Agile, ingestion, transformation et qualité des données, orchestration, modélisation ML, MLOps, déploiement et CI/CD. L'utilisation conjointe de Jira et GitHub permet de suivre les User Stories, les sous-tâches, les bugs, les Sprints, les branches et les Pull Requests.

# Chapitre 3

## Ingénierie des Données et DataOps

### 3.1 Introduction

Ce chapitre constitue le socle technique de notre projet de prédiction du décrochage universitaire. Il détaille la mise en œuvre de la première phase de la chaîne de valeur des données, structurée autour d’une approche **DataOps**. Nous verrons tout d’abord l’acquisition automatisée du jeu de données *xAPI-Edu-Data* et son ingestion dans notre base analytique DuckDB via l’outil `dlt`. Dans un second temps, nous présenterons la stratégie de transformation des données opérée avec `dbt` pour modéliser les variables clés du problème. Enfin, nous aborderons le dispositif complet de contrôle de la qualité, combinant contrats de données, validations techniques, règles métier et traçabilité pour garantir des données fiables aux futurs modèles de Machine Learning.

### 3.2 Description de la source de données

Le projet s’appuie sur le dataset *xAPI-Edu-Data* [1, 2], une source de données éducatives collectées via un système de gestion de l’apprentissage (LMS) utilisant le protocole xAPI.

Ce jeu de données se distingue par sa richesse comportementale, regroupant **480 enregistrements** d’étudiants définis par **16 indicateurs clés** :

- **Profils** : Genre, nationalité et niveau d’étude.
- **Comportements** : Levées de main, consultation des ressources, participation aux forums et aux annonces.
- **Absences** : Nombre de jours d’absence (inférieur ou supérieur à 7 jours).

**Objectif métier** : La variable cible classe les étudiants en trois niveaux de performance : **H** (High), **M** (Medium) et **L** (Low). Pour notre étude sur l’abandon universitaire, la classe **L** constitue notre indicateur prioritaire, car elle regroupe les profils à fort risque de décrochage scolaire.

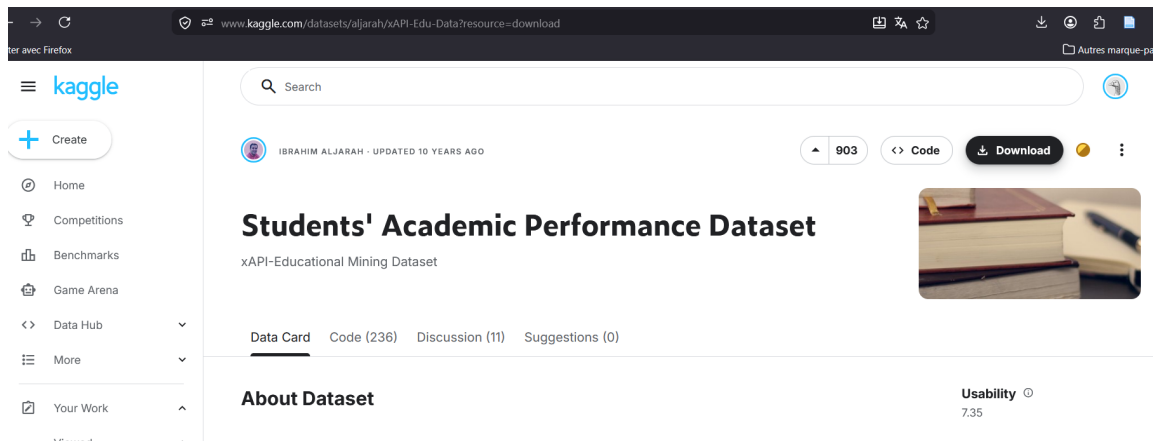


Figure 3.1 : Source des données sur la plateforme Kaggle

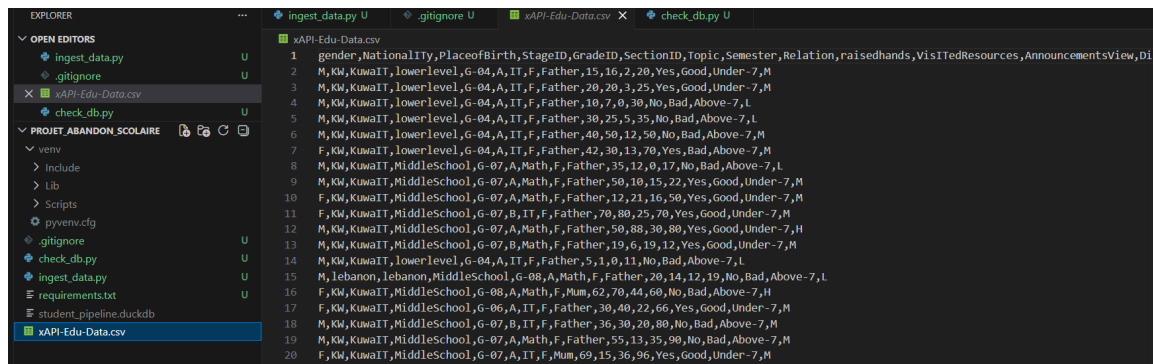


Figure 3.2 : Aperçu du jeu de données brut (format CSV)

### 3.3 Standardisation et reproductibilité de l'environnement

Afin d'assurer la stabilité et la collaboration au sein de l'équipe, nous avons mis en place une structure de travail professionnelle basée sur trois piliers :

- **Environnement Virtuel (venv)** : Création d'un espace isolé pour garantir que les versions des bibliothèques (pandas, dlt, duckdb) soient identiques pour tous les membres du groupe.
- **Gestion des Dépendances (requirements.txt)** : Génération d'un fichier listant l'intégralité des outils nécessaires, permettant un déploiement rapide de l'environnement sur n'importe quel poste.
- **Sécurité et Propreté (.gitignore)** : Configuration du contrôle de version pour exclure les fichiers volumineux (CSV, base DuckDB) et les fichiers système du dépôt GitHub, préservant ainsi la légèreté du projet.

### 3.4 Développement du pipeline d'ingestion avec dlt

Pour automatiser le transfert des données du format CSV vers notre base de données analytique, nous avons implémenté un pipeline d'ingestion à l'aide de l'outil `dlt` (Data Load Tool) [5]. Ce choix technique permet une gestion moderne et robuste de la donnée.

Le pipeline effectue trois actions clés :

1. **Normalisation des noms** : Les en-têtes originaux (ex : `raisedHands`) sont convertis en format standard SQL (`snake_case`, ex : `raised_hands`) pour éviter les erreurs lors des futures requêtes.
2. **Détection automatique du schéma** : `dlt` analyse le fichier CSV pour identifier les types de données (entiers, textes, etc.) sans configuration manuelle.
3. **Chargement structuré** : Les données sont injectées dans une table nommée `students_raw` au sein d'un schéma dédié `raw_data`.

```

4 def ingest_student_data():
5     # 1. Charger le fichier CSV avec pandas
6     # On précise le nom exact de ton fichier
7     file_path = "xAPI-Edu-Data.csv"
8     df = pd.read_csv(file_path)
9
10    # Nettoyage rapide des noms de colonnes (pour éviter les problèmes de SQL plus tard)
11    df.columns = [c.replace('-', '_').replace(' ', '_').lower() for c in df.columns]
12
13    # 2. Configurer le pipeline dlt
14    # Destination : duckdb | Nom de la base : 'student_data'
15    pipeline = dlt.pipeline(
16        pipeline_name="student_pipeline",
17        destination="duckdb",
18        dataset_name="raw_data"
19    )
20
21    # 3. Lancer l'ingestion vers la table 'students_raw'
22    load_info = pipeline.run(
23        df.to_dict(orient="records"),
24        table_name="students_raw",
25        write_disposition="replace"
26    )

```

Figure 3.3 : Script d'ingestion automatisé avec `dlt` et nettoyage des colonnes

### 3.5 Persistance et stockage des données (DuckDB)

L'ensemble des données ingérées est centralisé dans DuckDB [7], une base de données analytique (OLAP) performante et légère. Ce choix technologique permet de disposer d'une base locale ultra-rapide, idéale pour le prototypage et l'intégration avec `dbt`, sans la complexité d'un serveur externe. Les données sont organisées au sein d'un schéma nommé `raw_data`, créant ainsi une « Source de Vérité » brute. Ce stockage structuré garantit que les données restent immuables et prêtes à être consommées par les étapes de transformation et de modélisation.

```
(venv) PS C:\Users\ROMADI DOAA\OneDrive\Bureau\projet_abandon_scolaire> python ingest_data.py
--- Ingestion terminée avec succès ! ---
Pipeline student_pipeline load step completed in 1.14 seconds
1 load package(s) were loaded to destination duckdb and into dataset raw_data
The duckdb destination used duckdb:///C:\Users\ROMADI DOAA\OneDrive\Bureau\projet_abandon_scolaire\student_pipeline.duckdb location to store data
Load package 1786280425.282956 is LOADED and contains no failed jobs
```

Figure 3.4 : Confirmation du succès de l’ingestion et chargement des données dans DuckDB

## 3.6 Validation et contrôle de l’intégrité des données

En tant que DataOps, une étape cruciale consiste à vérifier la qualité des données après leur chargement. Pour cela, un script de test a été développé (`check_db.py`) permettant d’interroger directement la base DuckDB. Cette vérification confirme trois points essentiels :

1. **La présence de la table** : Confirmation que la table `students_raw` a bien été créée dans le schéma `raw_data`.
2. **L’intégrité des données** : Vérification que les 480 enregistrements sont bien présents.
3. **L’aperçu du contenu** : Validation visuelle des premières lignes pour s’assurer que les transformations (comme le format des noms de colonnes) sont correctes.

```
(venv) PS C:\Users\ROMADI DOAA\OneDrive\Bureau\projet_abandon_scolaire> python check_db.py
--- Tables dans la base ---
[]

--- Aperçu des données (5 lignes) ---
gender nationality placeofbirth stageid gradeid ... parentschoolsatisfaction studentabsencedays class _dlt load_id _dlt_id
0 M KW KuwaIT lowerlevel G-04 ... Good Under-7 M 1786280425.282956 I5d27Tc9Mg0/uw
1 M KW KuwaIT lowerlevel G-04 ... Good Under-7 M 1786280425.282956 gFed5ITkIWspVw
2 M KW KuwaIT lowerlevel G-04 ... Bad Above-7 L 1786280425.282956 RsFkA+PZvjx0g
3 M KW KuwaIT lowerlevel G-04 ... Bad Above-7 L 1786280425.282956 wYm8s2i6tAAlVA
4 M KW KuwaIT lowerlevel G-04 ... Bad Above-7 M 1786280425.282956 LWM/tt6KTbfXng
```

Figure 3.5 : Résultat du test de validation et aperçu des données stockées en base

## 3.7 La couche DataOps avec dbt

### 3.7.1 Objectif

Dans le cadre du projet *University Dropout Prediction*, la couche DataOps a pour objectif de préparer les données destinées aux étapes ultérieures de traitement et de prédiction. Cette contribution porte sur la mise en place d’une chaîne de transformation avec `dbt`, en s’appuyant sur les données déjà disponibles dans DuckDB.

L’utilisation de `dbt` [4] permet de structurer les transformations sous forme de modèles SQL, de formaliser les règles de préparation et d’intégrer des contrôles de qualité reproductibles. Cette organisation contribue à améliorer la fiabilité, la traçabilité et la maintenabilité des données utilisées dans le projet.

### 3.7.2 Conception de la chaîne de transformation

La chaîne de transformation repose sur une organisation en deux niveaux. Le premier niveau correspond à la préparation des données sources, tandis que le second regroupe les transformations destinées à produire les données utilisées dans la suite du projet.

La source `raw_data.students_raw` constitue le point de départ de cette chaîne. Elle est référencée dans le projet dbt afin de conserver une origine clairement identifiée pour les données. À partir de cette source, le modèle `stg_students` assure la préparation initiale des données.

Le modèle `prepared_students` constitue ensuite le niveau de préparation destiné aux traitements ultérieurs. Il s'appuie sur `stg_students`, ce qui permet de séparer les données sources des données transformées et de maintenir une dépendance claire entre les différentes étapes. Cette organisation peut être représentée comme suit :

```
raw_data.students_raw → stg_students → prepared_students
```

### 3.7.3 Préparation des variables

Les transformations réalisées dans `prepared_students` ont pour objectif de rendre certaines variables plus adaptées aux traitements ultérieurs.

La variable `studentabscondedays` a été transformée en une variable binaire appelée `absence_risk`. Cette transformation permet de représenter le niveau d'absence sous une forme numérique :

- `Under-7` : 0 (Équivalent à la valeur de référence)
- `Above-7` : 1 (Équivalent à la valeur transformée)

Ainsi, les catégories d'absence sont converties en une représentation simple et exploitable dans les traitements de données.

Une variable `risk_class` a également été créée à partir de `class`. Cette transformation permet de disposer d'une variable préparée correspondant à la classe de l'étudiant, tout en conservant une organisation cohérente avec les autres variables du modèle.

L'ensemble de ces transformations permet de produire une table préparée, structurée et adaptée aux besoins des étapes suivantes du projet.

### 3.7.4 Contrôle de qualité et validation

La qualité des données constitue un aspect essentiel de la chaîne DataOps. Dans cette contribution, des règles de contrôle ont été définies afin de vérifier la conformité des données préparées. Au total, **9 tests de qualité des données** ont été mis en place et sont définis dans `schema.yml` :

**Table 3.1** : Plan des tests de validation et de qualité

| Test | Colonne           | Rôle                                                                              |
|------|-------------------|-----------------------------------------------------------------------------------|
| 1    | gender            | Vérifie que la colonne ne contient pas de valeur nulle.                           |
| 2    | class             | Vérifie que chaque étudiant possède une valeur pour la classe.                    |
| 3    | raisedhands       | Vérifie l'absence de valeurs nulles dans le nombre de levées de main.             |
| 4    | visitedresources  | Vérifie que les informations concernant les ressources visitées sont renseignées. |
| 5    | announcementsview | Vérifie l'absence de valeurs nulles concernant les annonces consultées.           |
| 6    | discussion        | Vérifie que les informations relatives aux discussions sont renseignées.          |
| 7    | absence_risk      | Vérifie que la variable créée lors de la transformation n'est pas nulle.          |
| 8    | absence_risk      | Vérifie que absence_risk contient uniquement les valeurs autorisées 0 ou 1.       |
| 9    | risk_class        | Vérifie que la variable de risque associée à la classe n'est pas nulle.           |

Ces contrôles permettent principalement de garantir la complétude et la cohérence des données avant leur utilisation dans les étapes suivantes. Les résultats obtenus montrent que les **9 tests sont validés**, ce qui confirme que les données produites par les modèles respectent les règles de qualité définies.

### 3.7.5 Organisation des fichiers dbt

La partie DataOps réalisée avec dbt est organisée autour de plusieurs fichiers complémentaires, chacun ayant un rôle précis :

- Le fichier `dbt_project.yml` constitue le fichier principal de configuration du projet dbt. Il définit les paramètres généraux du projet ainsi que l'organisation des différents modèles dbt.
- Le fichier `sources.yml` permet de déclarer la source de données utilisée par dbt. Dans notre cas, il formalise la source `students_raw`, déjà disponible dans DuckDB, afin que les modèles dbt puissent l'utiliser de manière structurée et traçable.
- Le modèle `stg_students.sql` représente la première étape de transformation. Il permet de récupérer les données provenant de la source déclarée et de constituer une version de staging destinée aux traitements suivants.
- Le modèle `prepared_students.sql` réalise les transformations nécessaires pour préparer les données destinées aux étapes ultérieures du projet de prédiction.

- Le fichier `schema.yml` regroupe les descriptions des modèles ainsi que les règles de qualité appliquées aux colonnes.

## 3.8 Qualité des données

La qualité des données constitue une étape essentielle du projet de prédiction de l'abandon universitaire. Avant d'utiliser les données pour l'entraînement des modèles de Machine Learning, il est nécessaire de vérifier qu'elles sont complètes, cohérentes, valides et suffisamment fiables.

L'objectif de notre démarche Data Quality est donc de détecter les anomalies présentes dans les données et de distinguer les problèmes qui doivent bloquer le pipeline de ceux qui peuvent être signalés comme avertissements sans empêcher la poursuite du traitement.

La démarche a été appliquée à deux niveaux :

- les données **RAW**, directement issues de l'ingestion ;
- les données **PREPARED**, après transformation et préparation.

Cette séparation permet notamment de détecter les anomalies présentes dans les données originales avant qu'elles ne soient masquées ou supprimées par les transformations.

Notre démarche repose sur plusieurs composants complémentaires :

### 3.8.1 Data Contract

Le fichier `data_quality/data_contract.yml` définit les règles de qualité attendues pour les données. Il constitue une sorte de contrat entre les données et le pipeline. Il précise notamment :

- les colonnes obligatoires ;
- les colonnes qui ne doivent pas être nulles ;
- les valeurs autorisées ;
- les plages de valeurs numériques ;
- les règles d'unicité ;
- les règles d'unicité métier.

**Intérêt :** centraliser les contraintes dans un fichier de configuration permet de rendre les règles explicites, vérifiables et facilement maintenables.

```

20  columns:
21
22  gender:
23    type: string
24    nullable: false
25    allowed_values: ["M", "F"]
26
27  nationality:
28    type: string
29    nullable: false
30
31  placeofbirth:
32    type: string
33    nullable: false
34
35  stageid:
36    type: string
37    nullable: false
38    allowed_values:
39      - "lowerlevel"
40      - "MiddleSchool"
41      - "HighSchool"
42
43  gradeid:
44    type: string
45    nullable: false
46
47  sectionid:
48    type: string
49    nullable: false
50    allowed_values: ["A", "B", "C"]

```

Figure 3.6 : Extrait du schéma YAML de validation et de contrôle de conformité des données

### 3.8.2 Contrôles techniques — quality\_checks.py

Le fichier `data_quality/quality_checks.py` réalise les contrôles techniques sur les données RAW et PREPARED. Les principaux contrôles sont :

- **Complétude** : vérification des valeurs manquantes et du contenu du dataset.
- **Structure** : vérification de la présence des colonnes obligatoires.
- **Valeurs autorisées** : vérification des modalités des variables catégorielles.
- **Plages numériques** : détection de valeurs en dehors des intervalles attendus.
- **Doublons exacts** : recherche de lignes strictement identiques.
- **Unicité des identifiants** : vérification de l'unicité du `_dlt_id`.
- **Doublons métier** : recherche d'enregistrements ayant les mêmes caractéristiques métier.

**Intérêt** : ces contrôles permettent de détecter automatiquement les problèmes techniques susceptibles de compromettre les étapes suivantes du pipeline et notamment l'entraînement du modèle.

### 3.8.3 Règles métier — `check_business_rule.py`

Les contrôles techniques ne suffisent pas à garantir que les données ont un sens métier correct. Nous avons donc créé `data_quality/check_business_rule.py`. Deux règles principales sont vérifiées.

**Règle `absence_risk` :**

**Table 3.2 :** Règle métier du niveau d'absence des étudiants

| Nombre de jours d'absence  | <code>absence_risk</code> attendu | Interprétation du risque          |
|----------------------------|-----------------------------------|-----------------------------------|
| Under-7 (Moins de 7 jours) | 0                                 | Risque d'absence faible / normal  |
| Above-7 (7 jours ou plus)  | 1                                 | Risque d'absence élevé / critique |

**Règle `risk_class` :** La valeur de `risk_class` doit être cohérente avec la variable `class` : `risk_class = class`. Les deux règles doivent également respecter les contraintes de non-nullité.

**Intérêt :** ces contrôles permettent de vérifier que les variables calculées lors de la préparation des données respectent réellement la logique métier du projet. Une donnée peut être techniquement valide tout en étant incorrecte du point de vue métier.

### 3.8.4 Gestion des doublons métier

L'analyse a montré qu'il n'y avait :

- **0** doublon exact ;
- **0** `_dlt_id` dupliqué ;
- mais **4 lignes** appartenant à **2 groupes de doublons métier** dans les données RAW.

Pour cette raison, nous avons défini une clé métier basée sur les caractéristiques de l'étudiant et de son observation (`gender`, `nationality`, `placeofbirth`, `stageid`, `gradeid`, `sectionid`, `topic`, `semester`, `relation`, `raisedhands`, `visitedresources`, `announcementsview`, `discussion`, `parentansweringsurvey`, `parentschoolsatisfaction`, `studentanswereddays`, `class`).

Ce contrôle a permis d'identifier les 4 lignes impliquées dans 2 groupes de doublons métier. Les quatre lignes correspondent à deux groupes, chaque groupe contenant deux observations ayant des `_dlt_id` différents, ce qui montre l'intérêt de distinguer unicité technique et unicité métier.

Nous avons choisi de ne pas supprimer silencieusement les doublons dès l'étape RAW : ils sont conservés et signalés comme *warnings* au niveau RAW.

**Table 3.3 :** Bilan de l'exécution et de la qualité des données

| Contrôle                                   | Résultat           |
|--------------------------------------------|--------------------|
| Lignes RAW                                 | 480                |
| Colonnes                                   | 19                 |
| Complétude                                 | 100 %              |
| Doublons exacts                            | 0                  |
| IDs <code>_dlt_id</code> dupliqués         | 0                  |
| Lignes impliquées dans des doublons métier | 4                  |
| Groupes de doublons métier                 | 2                  |
| Violations <code>absence_risk</code>       | 0                  |
| Violations <code>risk_class</code>         | 0                  |
| Statut global                              | PASS_WITH_WARNINGS |

Le statut `PASS_WITH_WARNINGS` est volontaire. Il signifie que les données ne présentent aucune erreur bloquante, mais qu'un problème de qualité a été identifié et documenté : les deux groupes de doublons métier présents dans les données RAW.

```
=====
BUSINESS RULE CHECK
=====
Violations absence_risk : 0
Violations risk_class  : 0
Groupes doublons métier: 0
Statut                  : PASS
Toutes les règles métier sont respectées.
=====
```

**Figure 3.7 :** Résultat du contrôle de conformité métier

Les contrôles de qualité ont été complétés par les tests de données définis dans dbt :

```
1 dbt run PASS=2 ERROR=0
2 dbt test PASS=9 WARN=0 ERROR=0
```

**Intérêt :** cette décision permet de conserver une trace de la qualité réelle des données originales tout en permettant la poursuite du traitement.

### 3.8.5 Lineage des données

Le fichier `lineage.md` documente le parcours des données dans le projet :

```
xAPI-Edu-Data.csv → DLT → raw_data.students_raw → dbt →
main.stg_students → main.prepared_students → Machine Learning
```

La partie Data Quality intervient entre ces différentes étapes pour contrôler les données avant leur utilisation par le Machine Learning.

**Intérêt :** le lineage permet de savoir d'où viennent les données, quelles transformations elles subissent et à quel moment les contrôles de qualité sont appliqués. Il facilite ainsi la traçabilité et l'analyse des problèmes.

## Conclusion du chapitre

En alignant l'ingestion (dlt), la transformation (dbt) et la persistance (DuckDB) sur les standards DataOps, ce chapitre a permis d'établir une Source de Vérité robuste et entièrement reproductible. Les mécanismes de validation mis en place — combinant les tests dbt, les contrôles de qualité techniques et la vérification des règles métier — confirment une complétude de 100 % et un statut `PASS_WITH_WARNINGS` maîtrisé grâce au repérage de 2 groupes de doublons métier. Les données sont désormais nettoyées, documentées et parfaitement traçables, offrant un conteneur de données sain et prêt pour la phase de modélisation prédictive.

# Chapitre 4

## Modélisation Prédictive et MLOps

### 4.1 Introduction

Ce chapitre présente la phase de modélisation prédictive du risque de décrochage scolaire ainsi que son intégration dans un cycle de vie MLOps. Dans un premier temps, nous détaillons la démarche de Machine Learning, allant du benchmarking algorithmique à l'analyse fine des performances du modèle retenu (Random Forest), en mettant l'accent sur le rappel et l'interprétabilité métier. Dans un second temps, nous abordons l'industrialisation MLOps via MLflow, couvrant le tracking des expérimentations, le versionnement dans le Model Registry et la mise en place d'un monitoring continu capable d'alerter sur la dérive des données et les anomalies de pipeline.

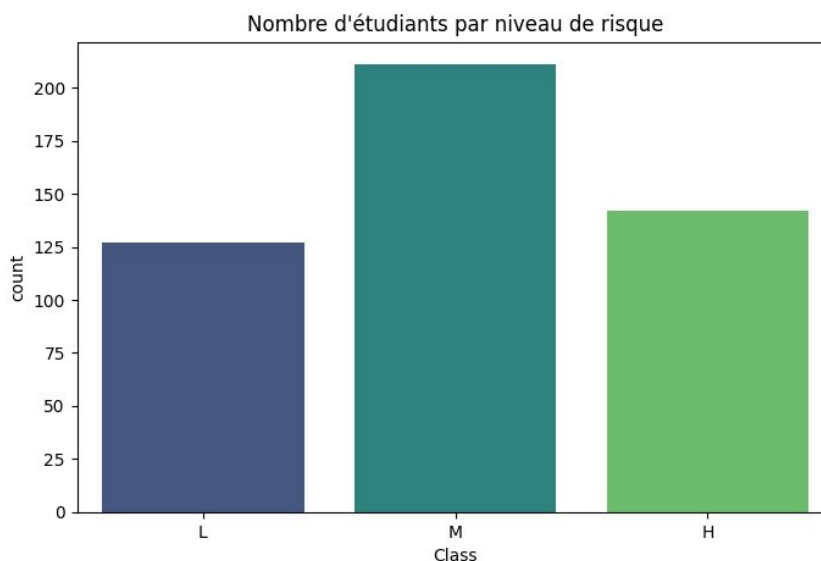
### 4.2 Analyse comparative et stratégie de sélection algorithmique

Le choix du moteur d'apprentissage constitue une étape centrale de notre démarche. Dans le cadre de ce projet, nous avons mené une étude comparative (*Benchmarking*) entre trois algorithmes : la **Régression Logistique**, le **Random Forest** et le **XGBoost**.

Les modèles ont été entraînés à partir des données préparées après ingestion et transformation, puis comparés sur l'ensemble de validation. La Régression Logistique obtient une Accuracy de **69,44 %**, tandis que le XGBoost atteint **80,56 %**. Le Random Forest obtient la meilleure performance avec une Accuracy de **81,94 %**.

Le Random Forest est donc sélectionné comme meilleur modèle sur la base de sa performance sur le jeu de validation. Ce résultat s'explique notamment par son adaptation aux données tabulaires et sa capacité à modéliser des relations non linéaires entre les différentes caractéristiques comportementales et académiques des étudiants.

## 4.3 Évaluation holistique de la performance



**Figure 4.1** : Répartition de la population étudiante par classe de risque académique

### 4.3.1 Le paradoxe de l'Accuracy dans le contexte éducatif

Dans le contexte de la prédiction du risque de décrochage, l'Accuracy constitue un indicateur important, mais elle ne permet pas à elle seule de décrire complètement le comportement du modèle. Une même Accuracy peut masquer des différences importantes dans la capacité du modèle à identifier correctement chacune des classes. C'est pourquoi nous avons également pris en compte la **Precision**, le **Recall** et le **F1-score** afin d'obtenir une évaluation plus complète. L'évaluation finale réalisée sur le jeu de test donne une Accuracy de **76 %**, ce qui permet d'estimer les performances du modèle sur des données qui n'ont pas été utilisées lors de l'entraînement.

### 4.3.2 La priorité au Rappel (Recall) : la sécurité avant tout

Dans un système de prédiction du risque étudiant, le Recall est particulièrement important pour identifier les étudiants appartenant réellement aux catégories à risque. Un faux négatif correspond à un étudiant appartenant réellement à une catégorie de risque élevée mais prédit dans une catégorie moins risquée. Cette situation peut conduire à ne pas identifier certains profils nécessitant un accompagnement.

Dans notre expérimentation, le Recall de la classe **High Risk** atteint **0,62**. Cette valeur constitue donc un élément important dans l'analyse finale du modèle et montre qu'une partie des étudiants réellement classés High est encore prédite comme une catégorie moins élevée.

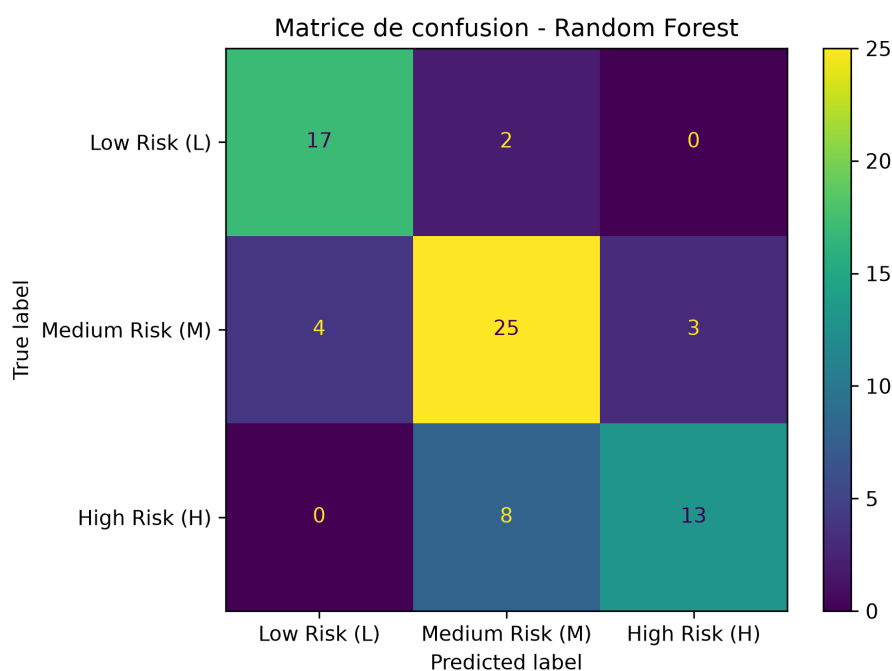
### 4.3.3 Précision et gestion des ressources : éviter la saturation

Un Recall élevé doit cependant être associé à une Precision suffisante. Un modèle générant trop de fausses alertes pourrait entraîner une utilisation inefficace des ressources d'accompagnement. L'objectif est donc de rechercher un équilibre entre la capacité à détecter les étudiants à risque et la fiabilité des alertes produites par le modèle. L'analyse conjointe de la Precision, du Recall et du F1-score permet ainsi de mieux caractériser les performances du Random Forest.

### 4.3.4 Le modèle comme outil de pilotage stratégique

Cette analyse permet d'envisager le modèle comme un outil d'aide à la décision. Les prédictions peuvent être utilisées pour identifier les profils nécessitant une attention particulière et pour orienter les actions d'accompagnement pédagogique. L'Accuracy de 76 % sur le jeu de test doit donc être interprétée conjointement avec les autres métriques et avec l'analyse de la matrice de confusion afin d'obtenir une vision plus complète du comportement du modèle.

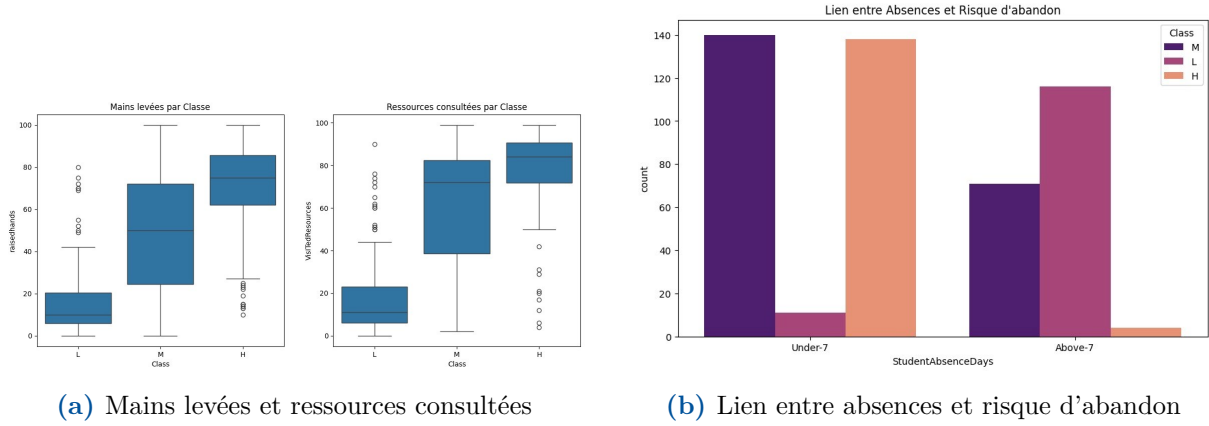
## 4.4 Diagnostic de robustesse et analyse des erreurs



**Figure 4.2 :** Matrice de confusion du modèle Random Forest sur l'ensemble de test

L'examen détaillé de la matrice de confusion permet d'analyser les erreurs réalisées par le modèle sur l'ensemble de test. Les résultats montrent que le modèle réalise correctement une grande partie des classifications, mais que certaines erreurs subsistent entre les différentes catégories de risque.

L'erreur particulièrement importante concerne la classe **High Risk** : plusieurs étudiants réellement classés High sont prédits Medium, ce qui conduit à une sous-estimation de leur niveau de risque.



**Figure 4.3 :** Analyse de l'engagement comportemental et de l'absentéisme

Grâce à l'analyse de l'importance des variables, nous pouvons identifier les facteurs qui contribuent le plus aux prédictions du Random Forest. Les variables **RaisedHands**, **VisitedResources** et **StudentAbsenceDays** figurent parmi les plus importantes. Cette analyse améliore l'interprétabilité du modèle, tout en rappelant qu'une variable importante n'est pas nécessairement une cause directe du décrochage.

## 4.5 Mise en œuvre MLOps : suivi, versionnement et monitoring

### 4.5.1 Contexte et objectifs

Le modèle utilisé (Random Forest) est entraîné par le module ML du projet à partir du dataset *xAPI-Edu-Data*, après transformation par le pipeline DataOps (ingestion, dbt, contrôle qualité). Sans outillage MLOps [12], chaque nouvel entraînement produirait un modèle sans trace de ses paramètres, de ses performances, ni de son historique — rendant impossible toute comparaison rigoureuse entre versions ou tout diagnostic en cas de problème en production.

L'objectif de cette partie était donc de répondre à trois besoins :

- **Traçabilité** : savoir à tout moment quel modèle a été entraîné, avec quels paramètres, et quelles performances ;
- **Reproductibilité** : pouvoir revenir à une version antérieure du modèle si nécessaire ;
- **Fiabilité en production** : détecter automatiquement si le modèle se dégrade dans le temps.

## 4.5.2 Architecture mise en place

```

1  ml/models/ (produits par l'équipe ML)
2  |
3  v
4  mlflow/log_to_mlflow.py --> Serveur MLflow (tracking + registry)
5  |
6  v
7  monitoring/model_monitor.py --> Detection de derive + metriques
8  |
9  v
10 monitoring/service_monitor.py --> Dashboard visuel (dashboard.
    png)

```

Le serveur MLflow est exécuté localement (`mlflow ui`), avec une base de stockage SQLite (`mlflow.db`). Cette architecture a ensuite été reprise et automatisée par l'équipe d'orchestration (Dagster), confirmant la pertinence de l'approche mise en place manuellement dans un premier temps.

## 4.5.3 Experiment Tracking

Le script `mlflow/log_to_mlflow.py` charge le modèle entraîné (`best_model.pkl`) ainsi que ses métadonnées (`model_version.json`, `feature_names.pkl`) et les enregistre dans MLflow [8] sous forme d'un run structuré.

**Paramètres enregistrés :** nom et version du modèle, dataset source, variable cible, date d'entraînement, métrique de sélection, statut de validation, nombre de variables (features).

**Table 4.1 :** Métriques enregistrées

| Métrique              | Valeur |
|-----------------------|--------|
| Validation accuracy   | 0.8194 |
| Test accuracy         | 0.76   |
| Test F1-score (macro) | 0.77   |

**Artefacts enregistrés :** matrice de confusion, fichier des noms de variables, fichier de métadonnées, et le modèle sérialisé lui-même.

Extrait significatif du script :

```

1  mlflow.set_tracking_uri("http://localhost:5001")
2  mlflow.set_experiment("university-dropout-prediction")
3
4  with mlflow.start_run(run_name="dropout-model-v1"):
5      mlflow.log_param("model_name", metadata.get("model_name"))

```

```

6 mlflow.log_metric("test_accuracy", metadata.get("test_accuracy"
   ))
7 mlflow.log_artifact("ml/evaluation/confusion_matrix.png")
8 mlflow.sklearn.log_model(model, "model")

```

#### 4.5.4 Model Registry — Versionnement

Une fois le run créé, le modèle est enregistré dans le MLflow Model Registry sous le nom `dropout-predictor`. Chaque nouvel enregistrement crée automatiquement une nouvelle version (v1, v2, v3...), permettant de conserver un historique complet des modèles produits au fil du projet, sans jamais écraser une version précédente.

```

1 run_id = mlflow.active_run().info.run_id
2 model_uri = f"runs://{run_id}/model"
3 mlflow.register_model(model_uri, "dropout-predictor")

```

Ce mécanisme a directement servi lors du ré-entraînement du modèle par l'équipe ML (suite à une correction du pipeline de préparation des données) : la nouvelle version a été enregistrée sous une version distincte (v3), sans perdre la trace des versions précédentes — démontrant concrètement l'intérêt du versionnement en conditions réelles de projet.

| Run Name         | Created     | Dataset | Duration | Source     | Models               |
|------------------|-------------|---------|----------|------------|----------------------|
| monitoring-check | 2 hours ago | -       | 102ms    | model_...  | -                    |
| dropout-model-v1 | 2 hours ago | -       | 31.0s    | log_to_... | dropout-predictor v3 |
| monitoring-check | 17 days ago | -       | 2.2s     | model_...  | -                    |
| monitoring-check | 17 days ago | -       | 114ms    | model_...  | -                    |
| monitoring-check | 17 days ago | -       | 85ms     | model_...  | -                    |
| monitoring-check | 19 days ago | -       | 100ms    | model_...  | -                    |
| monitoring-check | 20 days ago | -       | 2.3s     | model_...  | -                    |
| dropout-model-v1 | 20 days ago | -       | 31.6s    | log_to_... | dropout-predictor v2 |

**Figure 4.4 :** Historique des runs MLflow : suivi des expérimentations et versions successives du modèle enregistré dans le Model Registry (v2, v3)

#### 4.5.5 Monitoring du modèle

Le script `monitoring/model_monitor.py` simule la surveillance du modèle une fois en production :

- Rechargement du modèle depuis le Model Registry (et non directement depuis le fichier `.pkl`), garantissant l'utilisation de la version officiellement validée ;
- Calcul de l'accuracy et du F1-score sur un jeu de données représentant de « nouvelles » observations ;

- Détection de dérive (*data drift*) par comparaison des distributions statistiques entre les données d’entraînement et les nouvelles données ;
- Enregistrement des résultats dans MLflow pour constituer un historique consultable.

Le script `monitoring/service_monitor.py` complète ce dispositif en générant un dashboard visuel (`dashboard.png`) à partir de l’historique des runs de monitoring, représentant l’évolution de l’accuracy, du F1-score et de la dérive détectée dans le temps.

#### 4.5.6 Étude de cas : détection d’une anomalie de pipeline

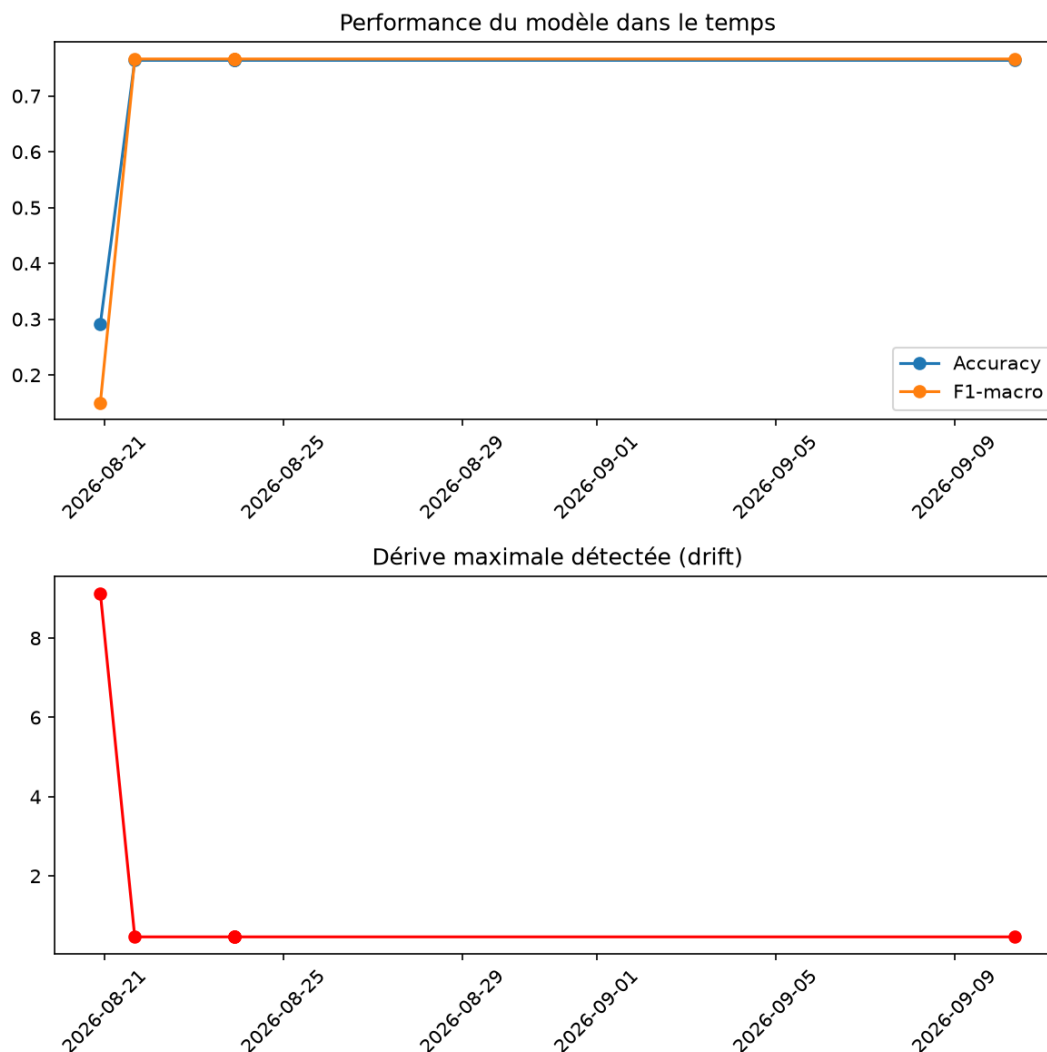
Lors des premiers tests du système de monitoring, une chute anormale de performance a été observée :

**Table 4.2 :** L’anomalie de pipeline

| Situation                                    | Accuracy |
|----------------------------------------------|----------|
| Accuracy attendue                            | 0.76     |
| Accuracy observée avant correction du scaler | 0.29     |

**Diagnostic :** le modèle avait été entraîné sur des données normalisées (Standard-Scaler), mais l’objet de normalisation (`scaler`) n’était pas sauvegardé par le pipeline de préprocessing. Le script de monitoring appliquait donc le modèle à des données brutes, non normalisées — provoquant une incohérence d’échelle entre les variables attendues par le modèle et celles réellement fournies.

**Résolution :** le problème a été signalé à l’équipe ML, qui a modifié le script de préprocessing pour sauvegarder le scaler (`ml/models/scaler.pkl`). Le script de monitoring a ensuite été mis à jour pour appliquer ce scaler avant toute prédiction, ramenant l’accuracy à **0.7639**, cohérente avec les performances attendues.



**Figure 4.5 :** Dashboard de monitoring du modèle : évolution de l’accuracy / F1-score et de la dérive maximale détectée dans le temps

#### 4.5.7 Intégration avec l’orchestration automatisée (Dagster)

La logique développée manuellement dans `mlflow/log_to_mlflow.py` a par la suite servi de référence à l’automatisation complète du pipeline via Dagster (`dataops/dagster/assets_ml.py`). Cette évolution démontre la transition du projet d’une approche MLOps manuelle et exploratoire vers un pipeline entièrement automatisé et reproductible, conforme aux bonnes pratiques MLOps visées par le projet.

Donc on a désormais automatiquement, à chaque exécution du pipeline :

- l’entraînement du modèle sur les données préparées ;
- l’enregistrement du run dans MLflow (paramètres, métriques, artefacts) ;
- la promotion conditionnelle dans le Model Registry, selon un seuil d’accuracy (0.75).

**Table 4.3 :** Difficultés rencontrées et solutions

| Difficulté                                                                           | Solution apportée                                                                         |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Scaler de normalisation non sauvegardé, faussant les prédictions du monitoring       | Signalement à l'équipe ML, sauvegarde du scaler, intégration dans le script de monitoring |
| Incompatibilité de version de pandas lors du chargement d'un fichier .pkl            | Mise à jour de pandas vers une version compatible                                         |
| Conflit de port (5000) réservé par Windows, empêchant le lancement du serveur MLflow | Reconfiguration du serveur et des scripts sur le port 5001                                |
| Blocage de sécurité Windows (Device Guard) sur l'exécutable mlflow.exe               | Contournement via <code>python -m mlflow ui</code>                                        |
| Dossiers experiments/ et registry/ prévus mais non utilisés                          | Nettoyage de la structure du dossier et mise à jour de la documentation                   |

## Conclusion du chapitre

L'expérimentation a permis de sélectionner un modèle Random Forest atteignant 76 % d'accuracy sur le jeu de test, tout en privilégiant la détection des profils à haut risque (High Risk). L'intégration de l'écosystème MLflow apporte la traçabilité et le versionnement indispensables pour piloter le cycle de vie du modèle. L'utilité concrète de cette couche MLOps s'est illustrée par la détection précoce d'une dérive due à l'absence du scaler, résolue avant la mise en production. Cet environnement d'expérimentation et de monitoring pose ainsi la référence nécessaire à l'automatisation globale du pipeline.

# Chapitre 5

## Orchestration de Données avec Dagster

### 5.1 Introduction

Dans un contexte de traitement de données ou de Machine Learning, l'**orchestration** désigne la coordination automatisée d'un ensemble d'étapes interdépendantes (ingestion, transformation, contrôle qualité, entraînement de modèle, etc.), en respectant leur ordre d'exécution, en gérant les dépendances entre elles, et en assurant leur déclenchement, leur suivi et leur traçabilité.

Un orchestrateur ne réalise pas lui-même la logique métier de chaque étape, il en pilote l'exécution. Avant la mise en place de Dagster, les différentes étapes du projet (ingestion via dlt, transformation via dbt, contrôles qualité, préparation des jeux d'entraînement, entraînement du modèle, journalisation dans MLflow) reposaient sur des scripts indépendants, à exécuter manuellement et dans le bon ordre, ce qui posait plusieurs problèmes :

- absence de garantie sur l'ordre d'exécution ;
- risque d'oubli d'une étape ;
- absence de traçabilité centralisée des exécutions ;
- aucune reprise automatique en cas d'échec ;
- nécessité d'une intervention humaine pour toute exécution récurrente.

Le script `run_pipeline.py` illustre bien cette limite, puisqu'il permet de matérialiser le pipeline de données en une seule commande mais doit être relancé manuellement à chaque fois, sans planification ni suivi avancé. La mise en place de Dagster répond ainsi à plusieurs objectifs :

- automatiser l'enchaînement complet ingestion → transformation → qualité → Machine Learning → MLflow ;
- garantir le respect des dépendances entre les étapes (une étape ne s'exécutant que si ses prérequis sont satisfaits) ;
- assurer la traçabilité de chaque exécution (lineage, logs, métadonnées) ;
- permettre une planification automatique (exécution quotidienne) ainsi qu'un déclenchement événementiel (arrivée d'un nouveau fichier de données) ;
- centraliser le suivi et la supervision du pipeline dans une interface unique.

## 5.2 Présentation de Dagster

Dagster [3] est un orchestrateur de données open source de type « asset-based » : plutôt que de modéliser un pipeline comme une simple succession de tâches, il le modélise comme un **graphe d'assets**, c'est-à-dire des objets de données persistants (table, fichier, modèle) produits par du code, et fournit une interface web permettant de visualiser ce graphe, de déclencher des exécutions, de consulter les logs et l'historique des runs, ainsi que des mécanismes de planification (*schedules*) et de déclenchement événementiel (*sensors*).

Le projet constitue fondamentalement une chaîne de transformation de données (données brutes → données transformées → jeux d'entraînement → modèle), à laquelle Dagster est particulièrement adapté puisqu'il permet de modéliser chaque étape comme un asset plutôt que comme une tâche isolée, avec plusieurs bénéfices exploités dans le projet :

- un **lineage visuel automatique** reliant chaque asset à ses dépendances amont et aval ;
- la possibilité de **rematérialiser** un asset donné et ses dépendants sans rejouer l'intégralité du pipeline ;
- l'attachement de **métadonnées** (nombre de lignes, taux de complétude, métriques du modèle, etc.) directement consultables dans l'interface pour chaque exécution.

Le projet contient, à titre de première itération, une définition alternative fondée sur des *op* et un *job* classique, non utilisée en production, le point d'entrée officiel du projet s'appuyant exclusivement sur les **Software-Defined Assets** décrits dans la suite de cette section.

Dagster occupe ainsi une position centrale dans l'architecture DataOps/MLOps du projet, coordonnant les modules développés indépendamment (ingestion, transformation, contrôle qualité, Machine Learning, suivi des expériences) sans en dupliquer la logique métier. Cette séparation des responsabilités entre Dagster pour l'orchestration et les modules métiers pour le traitement permet de garder un pipeline modulaire, maintenable et testable indépendamment de la couche d'orchestration.

## 5.3 Architecture de l'orchestration

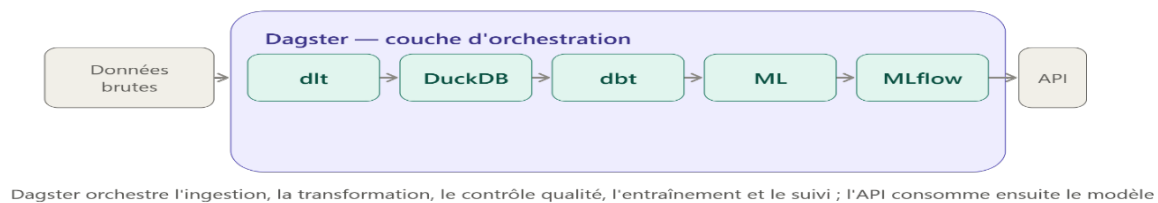
Le flux global du projet peut se résumer ainsi :

xAPI-Edu-Data → dlt → DuckDB → dbt → Machine Learning → MLflow → API

Dagster orchestre l'ensemble de ce flux depuis l'ingestion du fichier source jusqu'à la journalisation et l'évaluation du modèle dans MLflow : c'est lui qui déclenche successivement l'ingestion dlt, l'exécution dbt, les contrôles qualité, la préparation des jeux d'entraînement, l'entraînement du modèle et son enregistrement dans MLflow, en garantissant que chaque étape ne démarre qu'une fois la précédente terminée avec succès.

L'API de prédiction constitue la dernière brique du flux global, mais elle ne fait pas partie du graphe d'assets orchestré par Dagster : il s'agit d'un service indépendant, qui

charge directement au démarrage les artefacts sérialisés produits par le pipeline pour répondre aux requêtes de prédiction. Dagster orchestre donc la chaîne de bout en bout jusqu'à la production et la journalisation du modèle ; l'exposition de ce modèle via l'API est un processus de déploiement séparé, consommant les artefacts produits par le pipeline.



**Figure 5.1 :** Architecture globale du pipeline de données orchestré par Dagster

## 5.4 Organisation du pipeline sous Dagster

Dagster structure le pipeline autour de six composants complémentaires. Les **assets** représentent les objets de données produits à chaque étape (données brutes, données transformées, rapport de qualité, jeu d'entraînement, modèle entraîné, évaluation), reliés entre eux par des dépendances qui définissent l'ordre d'exécution. Les **ressources** centralisent la configuration et les connexions partagées, notamment l'accès à la base de données et au serveur de suivi des expériences.

Un **job** unique regroupe l'ensemble des assets à matérialiser, un **schedule** permet son déclenchement automatique selon une planification récurrente, et un **sensor** déclenche une exécution en réaction à l'arrivée de nouvelles données. L'ensemble de ces composants est assemblé au sein d'un point d'entrée unique regroupant assets, ressources, job, schedule et sensor.



**Figure 5.2 :** Graphe des assets Dagster du projet

## 5.5 Orchestration du pipeline Data et Machine Learning

La première étape du pipeline ingère le fichier de données source et le charge dans la base de données du projet, où les données sont stockées dans une table dédiée dont l'accès est centralisé via une ressource partagée. L'étape suivante exécute les transformations du projet, produisant une table de données préparées, qui sont ensuite validées par un asset dédié exécutant des contrôles de qualité (complétude, valeurs autorisées, plages numériques, unicité) ainsi qu'une vérification des règles métier. Ce n'est qu'une fois ce contrôle réussi que le jeu de données peut être consommé par la partie Machine Learning.

Cette dernière est orchestrée par des assets dédiés qui appellent le code déjà développé dans le projet sans le réécrire. Un premier asset produit les jeux d'entraînement, de validation et de test à partir des données préparées, en appliquant l'encodage des variables et la normalisation nécessaires. Un second entraîne plusieurs modèles candidats, sélectionne le meilleur selon la métrique retenue, puis calcule ses métriques de performance. Un dernier compare l'accuracy de test à un seuil de qualité afin de décider d'une éventuelle promotion du modèle. Le modèle retenu est sauvegardé localement, pour être consommé directement par l'API de prédiction, et dans MLflow, où il est journalisé puis, en cas de promotion, enregistré comme nouvelle version dans le Model Registry.

## 5.6 Intégration de MLflow

MLflow [8] est déployé comme un service indépendant du processus Dagster, exposant un serveur de suivi des expériences dont l'URI est fournie à Dagster via une ressource dédiée. Avant chaque entraînement, la disponibilité du serveur est vérifiée, puis un run MLflow est ouvert pour chaque matérialisation de l'asset d'entraînement, dans lequel sont journalisés les paramètres du modèle, les métriques de performance, ainsi que les artefacts associés. Le résultat de la décision de promotion est enregistré comme tag sur le run correspondant, et une nouvelle version du modèle est enregistrée dans le Model Registry si le seuil de qualité est atteint. Chaque matérialisation Dagster de l'asset d'entraînement correspond ainsi exactement à un run MLflow, dont l'identifiant est attaché comme métadonnée dans l'interface Dagster, ce qui permet de naviguer directement d'une exécution Dagster vers le run MLflow correspondant.

## 5.7 Configuration et automatisation du pipeline

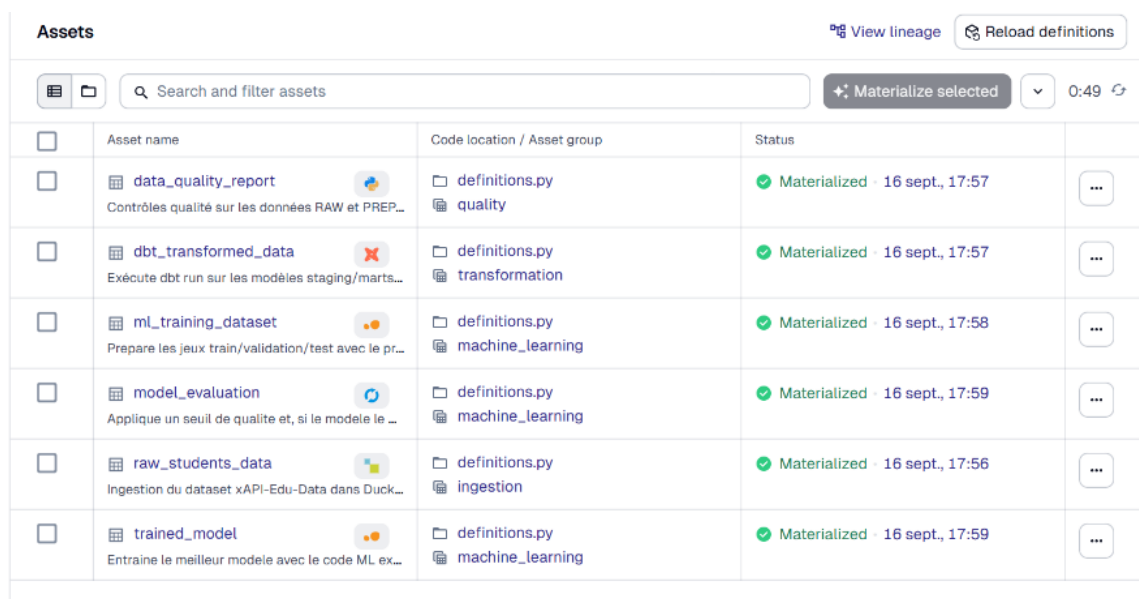
Le pipeline est piloté par un unique job Dagster, qui matérialise automatiquement l'ensemble des assets du graphe, y compris les assets Machine Learning. Un **schedule** planifie une exécution quotidienne du job, tôt le matin, tandis qu'un **sensor** surveille en continu le répertoire des données brutes et déclenche un nouveau run dès qu'un fichier supplémentaire y est détecté; ces deux mécanismes sont activés automatiquement dès le

démarrage du processus d'orchestration, un lancement manuel restant par ailleurs toujours possible.

Cette exécution s'appuie sur des **ressources** qui centralisent la configuration et les connexions partagées entre plusieurs assets, évitant toute duplication de code : une première ressource expose une connexion à la base de données partagée du projet, une seconde expose l'URI du serveur de suivi MLflow ainsi que le nom de l'expérience. Ces ressources sontinstanciées une seule fois au niveau des définitions du projet, ce qui permet de faire varier leur configuration selon l'environnement d'exécution sans modifier le code des assets.

## 5.8 Supervision et suivi des exécutions

L'ensemble du pipeline se pilote et se supervise depuis l'interface web de Dagster. Elle permet de lancer manuellement un run, de suivre en temps réel sa progression asset par asset, et de visualiser le graphe complet des dépendances regroupé par groupe fonctionnel. Elle permet également d'identifier immédiatement les échecs grâce au code couleur du graphe, de consulter les logs détaillés de chaque étape, ainsi que l'historique des exécutions passées et les métadonnées attachées à chaque asset.



| Assets                   |                                                                                    |                                    |                                  | <a href="#">View lineage</a> | <a href="#">Reload definitions</a> |
|--------------------------|------------------------------------------------------------------------------------|------------------------------------|----------------------------------|------------------------------|------------------------------------|
| <input type="checkbox"/> | Asset name                                                                         | Code location / Asset group        | Status                           |                              |                                    |
| <input type="checkbox"/> | <b>data_quality_report</b><br>Contrôles qualité sur les données RAW et PREP...     | definitions.py<br>quality          | ✓ Materialized · 16 sept., 17:57 |                              | ...                                |
| <input type="checkbox"/> | <b>dbt_transformed_data</b><br>Exécute dbt run sur les modèles staging/marts...    | definitions.py<br>transformation   | ✓ Materialized · 16 sept., 17:57 |                              | ...                                |
| <input type="checkbox"/> | <b>ml_training_dataset</b><br>Prepare les jeux train/validation/test avec le pr... | definitions.py<br>machine_learning | ✓ Materialized · 16 sept., 17:58 |                              | ...                                |
| <input type="checkbox"/> | <b>model_evaluation</b><br>Applique un seuil de qualité et, si le modèle le ...    | definitions.py<br>machine_learning | ✓ Materialized · 16 sept., 17:59 |                              | ...                                |
| <input type="checkbox"/> | <b>raw_students_data</b><br>Ingestion du dataset xAPI-Edu-Data dans Duck...        | definitions.py<br>ingestion        | ✓ Materialized · 16 sept., 17:56 |                              | ...                                |
| <input type="checkbox"/> | <b>trained_model</b><br>Entraîne le meilleur modèle avec le code ML ex...          | definitions.py<br>machine_learning | ✓ Materialized · 16 sept., 17:59 |                              | ...                                |

Figure 5.3 : État d'exécution et de matérialisation des assets Dagster

## Conclusion du chapitre

L'orchestration du projet repose sur un pipeline Dagster unique, modélisé sous forme de Software-Defined Assets, qui couvre de bout en bout la chaîne de traitement : ingestion des données brutes (dlt), transformation (dbt), contrôle qualité, préparation des jeux d'entraînement, entraînement du modèle et journalisation dans MLflow, puis évaluation et promotion conditionnelle dans le Model Registry. Associé à un job global, à un schedule

quotidien et à un sensor de détection de nouveaux fichiers, ce pipeline s'exécute de manière entièrement automatisée dès que le processus `dagster-daemon` est actif, tandis que l'intégration avec MLflow assure la traçabilité complète des expériences et la gestion versionnée du modèle destiné à être exposé par l'API de prédiction. Dagster constitue ainsi la brique centrale de l'architecture DataOps/MLOps du projet, assurant la coordination, la fiabilité et la supervision de l'ensemble du pipeline.

# Chapitre 6

## Déploiement REST, Conteneurisation et Industrialisation

### 6.1 Introduction

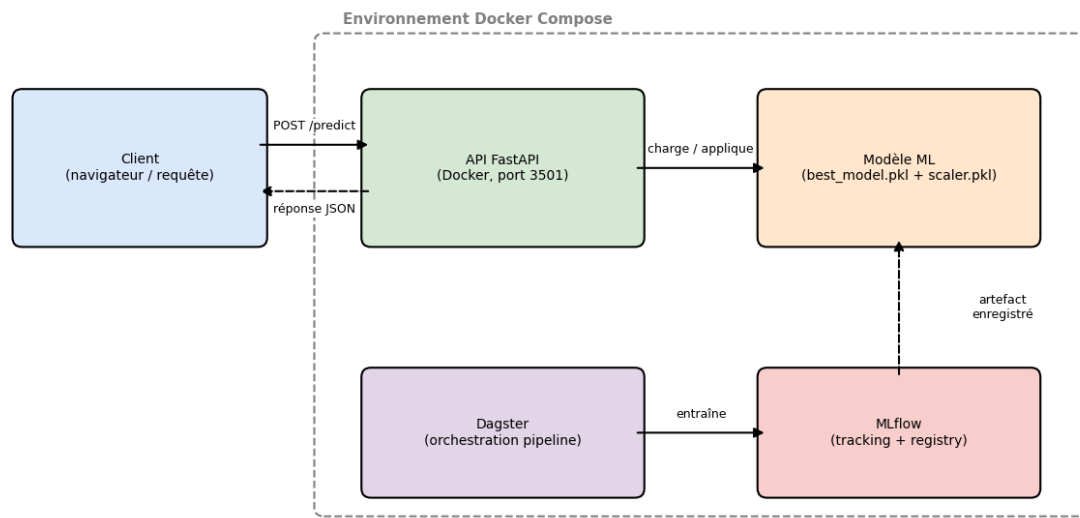
Ce chapitre détaille l'industrialisation du modèle prédictif et sa transformation en un service web de production accessible et conteneurisé. Nous présentons d'abord le développement de l'API REST avec FastAPI [9], gérant le prétraitement temps réel (StandardScaler, encodage) et la validation stricte des données via Pydantic. Nous abordons ensuite la conteneurisation et l'orchestration multi-services au moyen de Docker [6] et Docker Compose, l'exposition interactive de la documentation avec Swagger, ainsi que la validation continue du code (CI) exécutée par GitHub Actions. Enfin, nous décrivons le déploiement GitOps manuel sur la plateforme Komodo et la mise en place d'un monitoring d'infrastructure garantissant la disponibilité de l'API.

### 6.2 Architecture de déploiement

L'API repose sur trois artefacts produits en amont par le module d'entraînement (`ml/models/`) :

- `best_model.pkl` : le modèle Random Forest entraîné ;
- `scaler.pkl` : le StandardScaler utilisé lors de l'entraînement ;
- `feature_names.pkl` : la liste des 60 colonnes attendues par le modèle après encodage one-hot.

Le scaler doit impérativement être réappliqué à chaque nouvelle requête, afin que les données soumises au modèle en production respectent la même échelle que lors de l'apprentissage. L'ensemble du service est packagé sous forme de conteneur Docker, lui-même orchestré avec les autres services du projet (MLflow, Dagster) via Docker Compose, comme illustré à la Figure 6.1.



**Figure 6.1 :** Architecture générale du déploiement (API, MLflow, Dagster)

## 6.3 Conteneurisation de l'API avec Docker

### 6.3.1 Dockerfile

Le Dockerfile permet de construire une image Docker dédiée à l'API à partir de Python 3.11. Il installe uniquement les dépendances nécessaires au fonctionnement de l'API à partir du fichier `requirements-api.txt`, contrairement au `requirements.txt` global qui contient également Dagster, MLflow et dbt. Le Dockerfile copie ensuite le code de l'API ainsi que les artefacts du modèle entraîné, puis expose le port 8000. Enfin, le serveur Uvicorn est lancé afin de rendre l'API accessible.



```
deployment > Dockerfile
1 FROM python:3.11-slim
2 WORKDIR /app
3
4 # Installer uniquement les dependances necessaires a l'API
5 # (fichier allège, distinct de requirements.txt qui contient aussi
6 # dagster/mlflow/dbt pour les autres services)
7 COPY requirements-api.txt .
8 RUN pip install --no-cache-dir --default-timeout=120 -r requirements-api.txt
9
10 # Copier le code de l'API et les artefacts du modele
11 COPY api/ ./api/
12 COPY ml/models/ ./ml/models/
13 WORKDIR /app/api
14 EXPOSE 8000
15 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 6.2 : Dockerfile de l'API

### 6.3.2 Fichier de dépendances

Le fichier `requirements-api.txt` contient les bibliothèques nécessaires à l'API, notamment FastAPI, Uvicorn, pandas, scikit-learn et joblib. Les versions sont figées afin d'assurer la compatibilité avec le modèle `.pkl` et d'éviter les problèmes liés aux différences de versions lors de son chargement et de son exécution.

## 6.4 Orchestration avec Docker Compose

### 6.4.1 Lancement isolé de l'API

Le fichier `deployment/docker-compose.yml` permet de lancer uniquement le service API dans un conteneur Docker. Cette configuration est particulièrement utile pendant la phase de développement, pour tester rapidement les endpoints et vérifier le bon fonctionnement de l'API indépendamment des autres services du projet. Elle facilite également les tests avant le déploiement global de l'application.

### 6.4.2 Orchestration globale

Le fichier `docker-compose.yml`, situé à la racine du projet, permet d'orchestrer l'ensemble des services nécessaires à l'application. Il assure notamment le lancement et la communication entre l'API, le serveur MLflow, le webserver Dagster et le daemon Dagster. Cette configuration permet de démarrer les différents composants de manière centralisée et de garantir leur fonctionnement dans un environnement cohérent. Le port hôte **3501** a été choisi afin de correspondre au port attribué par le serveur Komodo, utilisé par le groupe pour le déploiement dans l'environnement cloud.

## 6.5 Implémentation de l'API

### 6.5.1 Point d'entrée

Le fichier `main.py` constitue le point d'entrée de l'application FastAPI. Il initialise l'application et rattache le routeur contenant les différents endpoints exposés par le service.

### 6.5.2 Schémas de validation

Les schémas Pydantic permettent de définir et de valider la structure des données échangées avec l'API. `StudentData` représente les données reçues en entrée, tandis que `PredictionResponse` définit le format de la réponse retournée par le service. Le schéma `StudentData` contient les seize variables décrivant le profil de l'étudiant, notamment les informations démographiques, la participation en classe, l'implication des parents et les absences.

### 6.5.3 Endpoints

L'API expose deux endpoints principaux :

- GET `/health` : vérifie que le service est disponible et fonctionne correctement.
- POST `/predict` : reçoit les données d'un étudiant et retourne son niveau de risque.

Avant la prédiction, les variables catégorielles sont transformées en one-hot encoding. Les données sont ensuite alignées sur les 60 colonnes utilisées lors de l'entraînement, puis normalisées avec le même scaler afin de conserver le même traitement que lors de l'entraînement du modèle. La prédiction retournée correspond à l'un des trois niveaux : **L** (Low Risk), **M** (Medium Risk) ou **H** (High Risk).

```
56
57 @router.post("/predict", response_model=PredictionResponse)
58 def predict(data: StudentData):
59     try:
60         X = preprocess_input(data)
61         X_scaled = scaler.transform(X)
62         pred = model.predict(X_scaled)[0]
63
64         return PredictionResponse(
65             prediction=CLASS_LETTERS[pred],
66             risk_label=RISK_LABELS[pred]
67         )
68     except Exception as e:
69         raise HTTPException(status_code=500, detail=str(e))
```

Figure 6.3 : Extrait du code de l'endpoint `/predict` (`api/routes.py`)

## 6.6 Lancement et test du service

### 6.6.1 Lancement en local

Depuis la racine du projet, le service complet est lancé via Docker Compose. L'API est alors accessible sur `http://localhost:3501`, avec une documentation interactive Swagger accessible sur `http://localhost:3501/docs`.

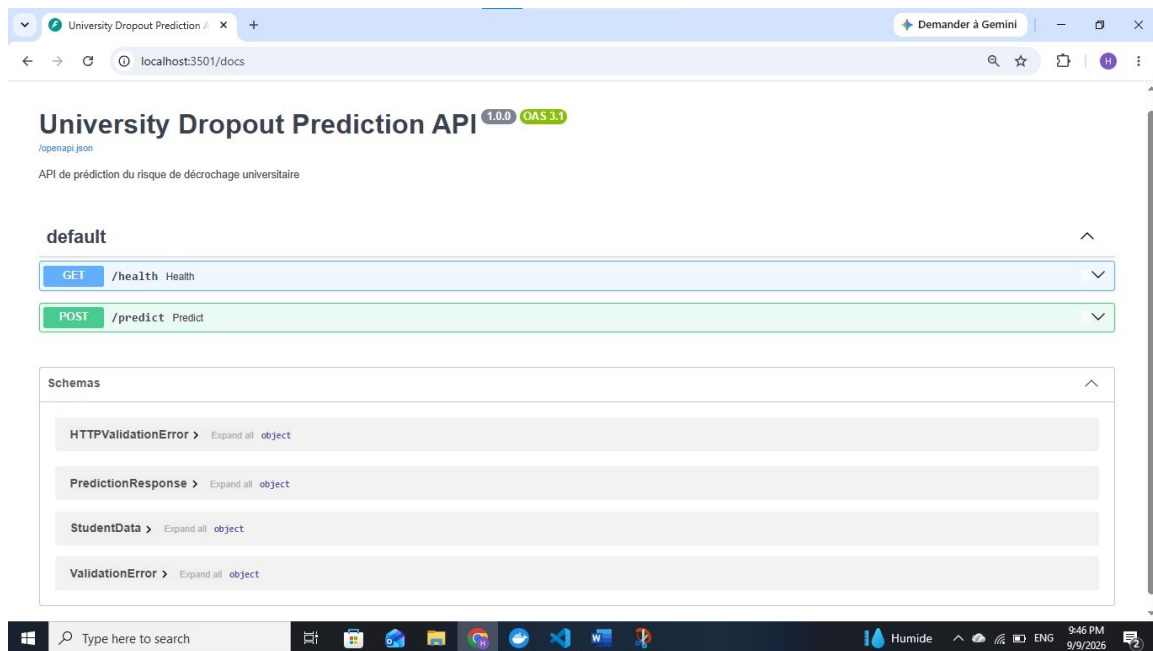


Figure 6.4 : Documentation interactive Swagger de l'API

### 6.6.2 Test et validation

Avant son intégration dans le pipeline global, le service a été testé manuellement à travers l'interface Swagger, en soumettant plusieurs profils d'étudiants représentatifs des trois niveaux de risque (faible, moyen, élevé). Ces tests ont permis de vérifier que les prédictions renvoyées étaient cohérentes avec les résultats obtenus lors de l'évaluation du modèle, et que le service répondait correctement même avec des données incomplètes ou incohérentes, grâce à la validation automatique des schémas Pydantic. L'endpoint `/health` a également été utilisé pour confirmer la disponibilité continue du service pendant les phases de test, notamment après un redémarrage du conteneur.

```
(base) PS C:\Users\Probook 430 G3\Documents\university-dropout-prediction> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                               NAMES
fd739cddaaa2   university-dropout-prediction-dagster-webserver  "dagster-webserver -..." 8 days
ago Up About an hour 0.0.0.0:3502->3000/tcp, [::]:3502->3000/tcp university-dropout-predict
ion-dagster-webserver-1
acc915fc99cf   university-dropout-prediction-api              "uvicorn main:app -..." 8 days
ago Up About an hour 0.0.0.0:3501->8000/tcp, [::]:3501->8000/tcp university-dropout-predict
ion-api-1
ac93714789f0   university-dropout-prediction-mlflow           "mlflow server --hos..." 8 days
ago Up About an hour 5000/tcp                university-dropout-predict
ion-mlflow-1
c3e2b33e89c2   university-dropout-prediction-dagster-daemon    "dagster-daemon run ..." 8 days
ago Up About an hour 3000/tcp                university-dropout-predict
ion-dagster-daemon-1
(base) PS C:\Users\Probook 430 G3\Documents\university-dropout-prediction> 
```

Figure 6.5 : Conteneurs Docker en cours d'exécution

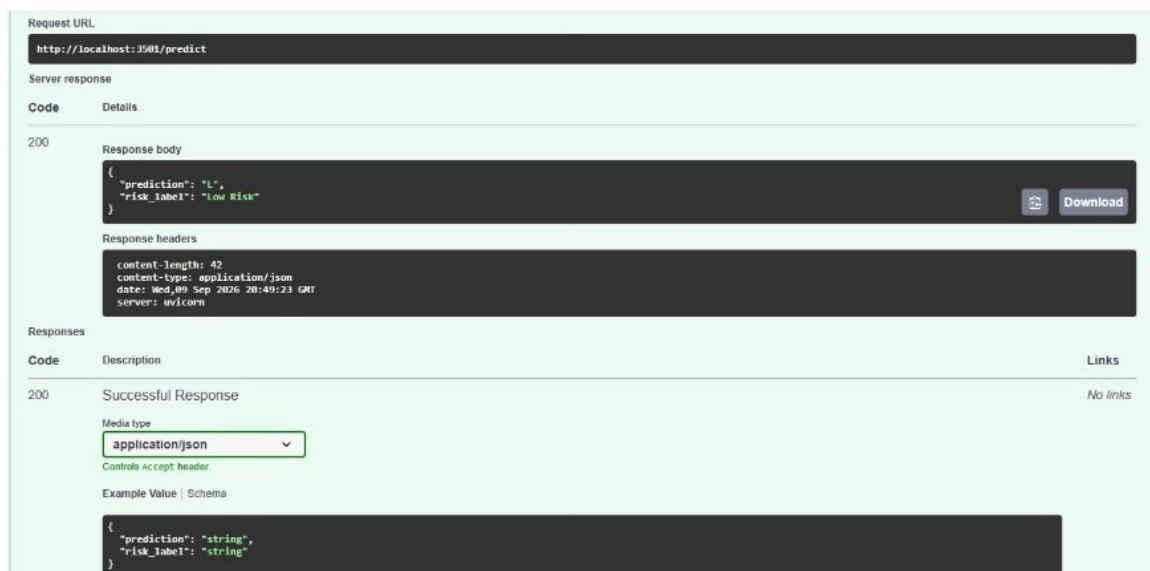


Figure 6.6 : Réponse de l'endpoint /predict dans Swagger

## 6.7 Choix techniques

La séparation entre `requirements-api.txt` et `requirements.txt` permet de garder une image Docker légère et de réduire le temps de construction du conteneur. Pour garantir des prédictions cohérentes, le même `StandardScaler` utilisé lors de l'entraînement est réutilisé en production. Les versions des bibliothèques sont également fixées afin d'éviter les problèmes de compatibilité avec le modèle `.pkl`. Enfin, le port 3501 a été choisi pour correspondre à la configuration du serveur Komodo utilisé pour le déploiement cloud.

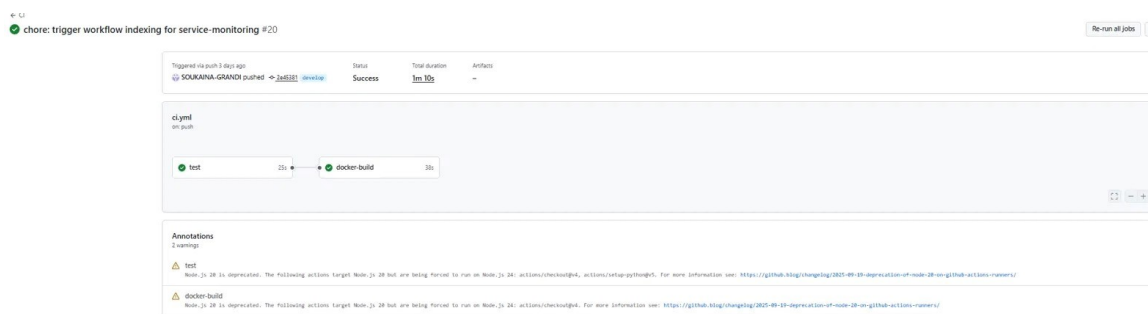
## 6.8 Intégration Continue (CI)

Une chaîne d'intégration continue a été mise en place avec GitHub Actions, se déclenchant automatiquement à chaque envoi de code vers le dépôt du projet. Elle exécute,

dans un environnement standardisé, une suite de tests automatisés couvrant la disponibilité du service (endpoint `/health`), une requête de prédiction valide, ainsi que le rejet d'une requête mal formée, avant de construire et de vérifier l'image Docker de l'API.

**Table 6.1** : Bilans temporels et statuts du pipeline

| Étape du pipeline                       | Résultat | Durée observée |
|-----------------------------------------|----------|----------------|
| Vérification et tests                   | Succès   | ≈ 25 secondes  |
| Construction et vérification de l'image | Succès   | ≈ 38 secondes  |
| Exécution complète du pipeline          | Succès   | ≈ 1 minute 10  |

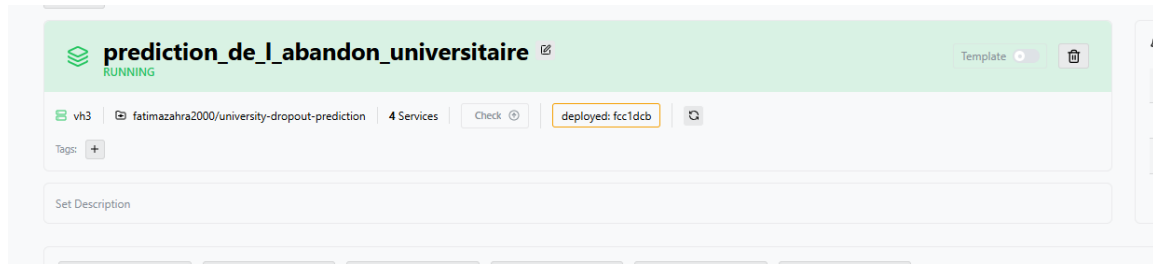


**Figure 6.7** : Exécution réussie du pipeline d'intégration continue (GitHub Actions)

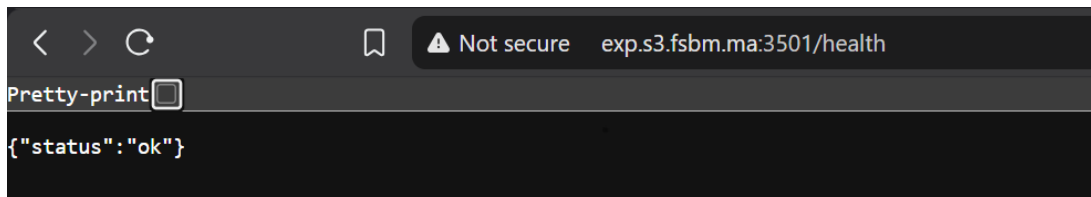
## 6.9 Déploiement sur Komodo

Le déploiement du service est assuré par **Komodo**, une plateforme auto-hébergée de gestion de conteneurs fonctionnant selon une approche **GitOps** : la définition de la pile applicative est récupérée automatiquement depuis le dépôt de code, sans copie manuelle de fichiers sur le serveur cible. Conformément aux consignes pédagogiques du module, le déclenchement du déploiement reste exclusivement manuel : aucune mise à jour automatique du service en production n'est déclenchée par un envoi de code. Chaque déploiement correspond à une action volontaire, effectuée après vérification de la stabilité du code à déployer.

Le déploiement a été réalisé avec succès sur l'infrastructure cible : l'ensemble des services de la pile applicative sont opérationnels, et le bon fonctionnement de l'API a été vérifié à la fois en interne et depuis un réseau externe, conformément à la convention d'accès définie pour l'infrastructure partagée.



**Figure 6.8 :** État du déploiement sur la plateforme Cloud : services opérationnels (statut RUNNING)



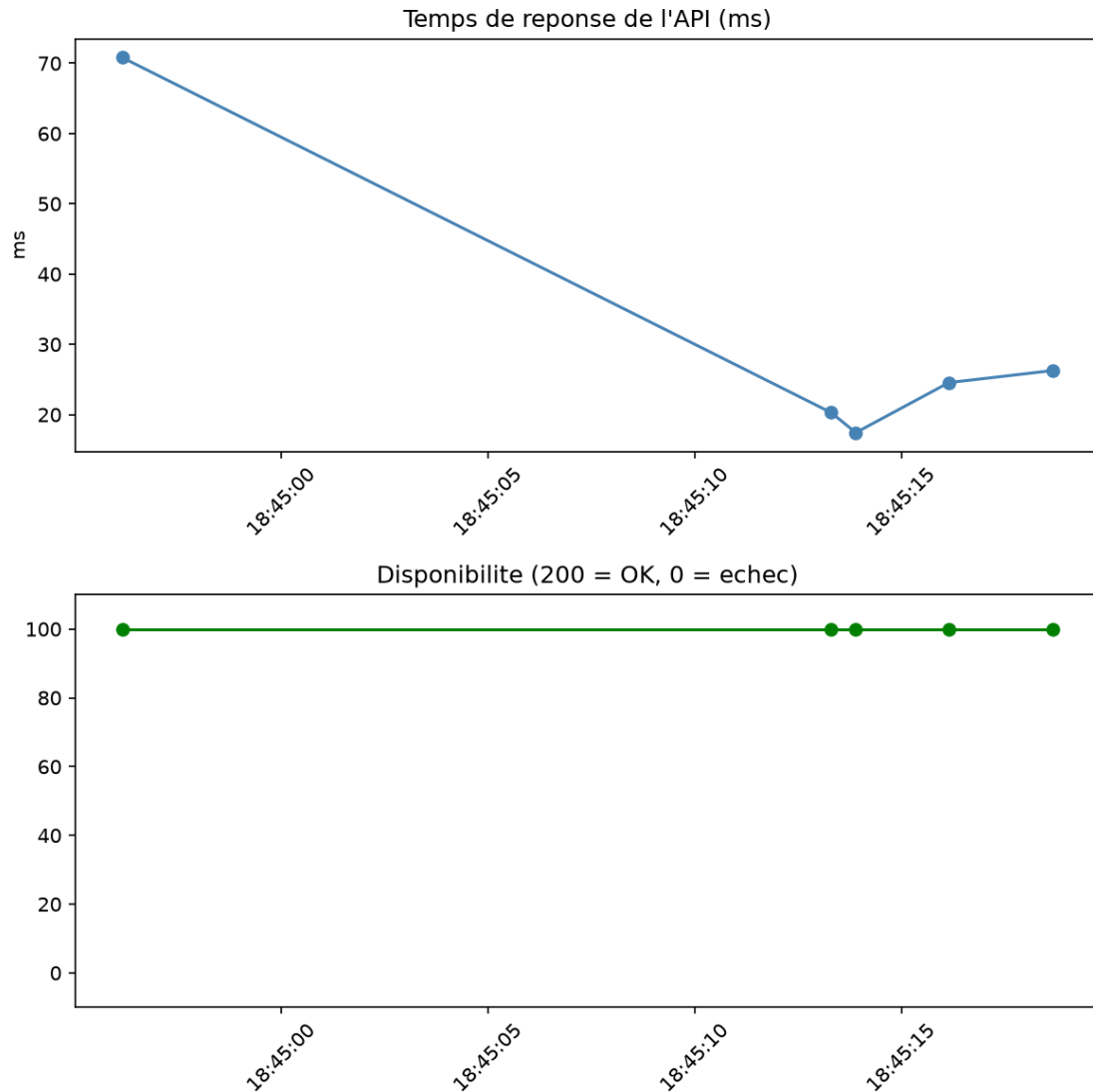
**Figure 6.9 :** Vérification du bon fonctionnement du service depuis un réseau externe

## 6.10 Monitoring du service

Un dispositif de suivi distinct du suivi des performances du modèle a été mis en place afin d’observer la disponibilité, le temps de réponse et les erreurs éventuelles du service une fois déployé. Un script interroge périodiquement l’endpoint de santé et journalise chaque vérification ; un second script produit à partir de cet historique un tableau de bord visuel et un résumé chiffré.

**Table 6.2 :** Synthèse des performances système

| Indicateur             | Valeur mesurée |
|------------------------|----------------|
| Disponibilité          | 100 %          |
| Temps de réponse moyen | ≈ 32 ms        |



**Figure 6.10 :** Évolution du temps de réponse et de la disponibilité du service

L'ensemble des choix de configuration relatifs au pipeline CI/CD, à la plateforme de déploiement et au dispositif de monitoring a fait l'objet d'une documentation d'architecture dédiée, conservée dans le dépôt du projet, afin de permettre à l'équipe de comprendre et de faire évoluer cette configuration de façon autonome.

## Conclusion du chapitre

L'ensemble de ce chapitre démontre le passage réussi du modèle d'expérimentation à un service web de production prêt à l'emploi. Grâce à la conteneurisation Docker, l'interfaçage FastAPI / Swagger et les tests automatisés GitHub Actions, l'API assure un traitement fiable des requêtes avec une validation stricte des entrées. L'intégration sur Komodo selon une approche GitOps sécurisée garantit un déploiement cloud maîtrisé. Avec une disponibilité mesurée à 100 % et un temps de réponse moyen de 32 ms, l'infrastructure offre une base robuste, documentée et prête à être consommée par les applications clientes.

# Conclusion générale et perspectives

## Bilan du projet

Le projet **BrainOps** avait pour ambition de concevoir, développer et déployer un système automatisé de prédiction du risque de décrochage universitaire, fondé sur l'analyse de données comportementales d'apprentissage. Au terme de ce travail, nous pouvons dresser un bilan globalement positif, tant sur le plan technique que méthodologique.

Sur le plan **technique**, le projet a permis de mettre en œuvre une architecture complète de type MLOps [12], couvrant l'ensemble du cycle de vie de la donnée et du modèle :

- **Ingestion** automatisée des données brutes via `dlt` ;
- **Stockage** dans une base analytique DuckDB ;
- **Transformation** et **contrôle qualité** avec `dbt` et des scripts Python dédiés ;
- **Modélisation** par apprentissage supervisé, avec comparaison de plusieurs algorithmes (Logistic Regression, Random Forest, XGBoost) ;
- **Suivi des expérimentations** et **versionnement** des modèles avec `MLflow` ;
- **Orchestration** complète du pipeline avec `Dagster` ;
- **Déploiement** du modèle sous forme d'API REST avec `FastAPI` et `Docker` ;
- **Intégration continue** et **déploiement** avec `GitHub Actions` et `Komodo`.

Sur le plan **méthodologique**, la démarche Agile Scrum a permis de structurer le travail en trois Sprints complémentaires, avec une gestion rigoureuse du backlog, des User Stories et des bugs via `Jira`, ainsi qu'une collaboration efficace via `GitHub`.

## Résultats obtenus

Les principaux résultats obtenus sont les suivants :

- Un modèle **Random Forest** atteignant **81,94 %** d'accuracy sur le jeu de validation et **76 %** sur le jeu de test ;
- Un pipeline de données entièrement automatisé, avec un statut de qualité `PASS_WITH_WARNINGS` documenté ;
- Une API REST accessible et documentée, avec une disponibilité mesurée à **100 %** et un temps de réponse moyen de **32 ms** ;
- Une intégration MLOps complète, avec traçabilité des expériences et versionnement des modèles.

## Limites du projet

Plusieurs limites peuvent être identifiées :

- Le **Recall** de la classe *High Risk* reste modéré (**0,62**), ce qui signifie qu'une partie des étudiants réellement à haut risque n'est pas détectée ;
- Le jeu de données utilisé (*xAPI-Edu-Data*) est relativement petit (**480 enregistrements**) et ne reflète pas nécessairement la diversité des situations réelles ;
- Le déploiement sur Komodo reste **manuel**, conformément aux consignes pédagogiques, ce qui limite le niveau d'automatisation ;
- L'absence de données longitudinales ne permet pas de suivre l'évolution des étudiants dans le temps.

## Perspectives d'amélioration

Plusieurs pistes d'amélioration peuvent être envisagées :

1. **Améliorer le rappel de la classe High Risk** en utilisant des techniques de rééchantillonnage (SMOTE, class weighting) ou des algorithmes adaptés aux classes déséquilibrées ;
2. **Enrichir le jeu de données** avec des données longitudinales et des indicateurs supplémentaires (fréquentation, résultats intermédiaires, etc.) ;
3. **Mettre en place un vrai monitoring en production** avec des alertes automatiques en cas de dérive ou de baisse de performance ;
4. **Automatiser davantage le déploiement** avec une approche GitOps complète (ArgoCD, Flux) ;
5. **Développer une interface utilisateur** (dashboard) destinée aux responsables pédagogiques, facilitant l'exploitation des prédictions ;
6. **Explorer des modèles plus avancés** (deep learning, modèles interprétables) et comparer leurs performances.

## Mot de fin

En conclusion, le projet BrainOps a permis de démontrer la faisabilité et l'intérêt d'une approche MLOps complète appliquée à la prédiction du décrochage universitaire. Au-delà des résultats techniques obtenus, ce projet a été une expérience formatrice enrichissante, tant sur le plan de la collaboration en équipe que sur celui de la mise en œuvre concrète d'un pipeline de données et de Machine Learning de bout en bout. Les compétences acquises — en Data Engineering, Machine Learning, MLOps, DevOps et gestion de projet Agile — constituent une base solide pour nos futurs projets professionnels.

# Annexe A

## Accès au dépôt GitHub du projet

L'ensemble du code source, des notebooks, des configurations et de la documentation du projet **BrainOps** est disponible publiquement sur GitHub.

### Dépôt GitHub :

<https://github.com/fatimazahra2000/university-dropout-prediction>

### Contenu du dépôt :

- Le pipeline d'ingestion des données (**dlt**, **DuckDB**) ;
- Les modèles de transformation **dbt** et les contrôles de qualité ;
- Le code d'entraînement des modèles de Machine Learning ;
- Les scripts MLOps (**MLflow**, **monitoring**) ;
- L'API de prédiction (**FastAPI**) et sa conteneurisation (**Docker**) ;
- L'orchestration du pipeline avec **Dagster** ;
- La documentation technique et le **README** du projet.

### Clone du dépôt :

```
1 git clone https://github.com/votre-utilisateur/university-  
  dropout-prediction.git  
2 cd university-dropout-prediction
```

# Annexe B

## Structure du dépôt GitHub

```
1  university-dropout-prediction/
2  |-- .github/
3  |   '-- workflows/
4  |       '-- ci.yml
5  |-- api/
6  |   |-- README.md
7  |   |-- main.py
8  |   |-- routes.py
9  |   '-- schemas.py
10 |-- data/
11 |   |-- DuckDB/
12 |   |-- raw/
13 |   '-- processed/
14 |-- data_quality/
15 |   |-- README.md
16 |   |-- check_business_rule.py
17 |   |-- data_contract.yaml
18 |   |-- lineage.md
19 |   '-- quality_checks.py
20 |-- dataops/
21 |   |-- dagster/
22 |   |   |-- Dockerfile
23 |   |   |-- README.md
24 |   |   |-- assets.py
25 |   |   |-- assets_ml.py
26 |   |   |-- definitions.py
27 |   |   |-- jobs_schedules.py
28 |   |   '-- resources.py
29 |   |-- dbt/
30 |   |   |-- university_dropout_dbt/
31 |   |   |   |-- analyses/
32 |   |   |   |-- macros/
33 |   |   |   |-- models/
34 |   |   |   |   |-- prepared_students.sql
35 |   |   |   |   |-- schema.yml
36 |   |   |   |   |-- sources.yml
37 |   |   |   |   '-- stg_students.sql
38 |   |   |   |-- seeds/
39 |   |   |   |-- snapshots/
40 |   |   |   |-- tests/
41 |   |   |   |-- README.md
42 |   |   |   '-- dbt_project.yml
```

```

43 | | |-- README.md
44 | | '-- profiles.yml
45 | '-- dlt/
46 | |-- README.md
47 | |-- check_db.py
48 | '-- ingest_data.py
49 |-- deployment/
50 | |-- Dockerfile
51 | |-- README.md
52 | '-- docker-compose.yml
53 |-- docs/
54 | |-- agile/
55 | |-- architecture/
56 | '-- rapport/
57 |-- ml/
58 | |-- evaluation/
59 | | |-- confusion_matrix.png
60 | | '-- evaluate_model.py
61 | |-- models/
62 | | |-- best_model.pkl
63 | | |-- feature_names.pkl
64 | | |-- model_version.json
65 | | '-- scaler.pkl
66 | |-- notebooks/
67 | | '-- EDA_Visualisation.ipynb
68 | |-- preprocessing/
69 | | '-- preprocess.py
70 | '-- training/
71 | '-- README.md
72 |-- mlflow/
73 | |-- Dockerfile
74 | |-- README.md
75 | '-- log_to_mlflow.py
76 |-- monitoring/
77 | |-- README.md
78 | |-- api_health_monitor.py
79 | |-- dashboard.png
80 | |-- model_monitor.py
81 | |-- service_dashboard.png
82 | |-- service_dashboard.py
83 | |-- service_metrics.csv
84 | '-- service_monitor.py
85 |-- tests/
86 |-- .env.example
87 |-- .gitignore
88 |-- README.md
89 |-- docker-compose.yml
90 |-- requirements-api.txt
91 |-- requirements.txt
92 '-- run_pipeline.py

```

# Annexe C

## Extrait du fichier docker-compose.yml

```
1  version: '3.8'
2
3  services:
4    api:
5      build:
6        context: .
7        dockerfile: deployment/Dockerfile
8      ports:
9        - "3501:8000"
10     volumes:
11       - ./ml/models:/app/ml/models
12     depends_on:
13       - mlflow
14     networks:
15       - brainops-network
16
17     mlflow:
18       image: ghcr.io/mlflow/mlflow:latest
19       ports:
20         - "5001:5000"
21       command: >
22         mlflow server
23         --host 0.0.0.0
24         --port 5000
25         --backend-store-uri sqlite:///mlflow.db
26       networks:
27         - brainops-network
28
29     dagster-webserver:
30       build: .
31       ports:
32         - "3502:3000"
33       command: dagster-webserver -h 0.0.0.0 -p 3000
34       networks:
35         - brainops-network
36
37     dagster-daemon:
```

```
38   build: .
39   command: dagster-daemon run
40   networks:
41   - brainops-network
42
43   networks:
44   brainops-network:
45   driver: bridge
```

# Annexe D

## Commandes utiles

```
1  # Lancer le pipeline d'ingestion
2  python scripts/ingest_data.py
3
4  # Verifier la base DuckDB
5  python scripts/check_db.py
6
7  # Executer les transformations dbt
8  cd dataops/dbt && dbt run
9
10 # Executer les tests dbt
11 dbt test
12
13 # Lancer le serveur MLflow
14 python -m mlflow ui --port 5001
15
16 # Lancer Dagster
17 dagster dev -f dataops/dagster/definitions.py
18
19 # Lancer l'API en local
20 uvicorn api.main:app --host 0.0.0.0 --port 8000
21
22 # Lancer tous les services avec Docker Compose
23 docker-compose up -d
24
25 # Verifier l'etat des conteneurs
26 docker ps
27
28 # Tester l'endpoint /health
29 curl http://localhost:3501/health
```

# Bibliographie

- [1] Ibrahim Aljarah. Students' academic performance dataset, 2016.
- [2] Elaf Abu Amrieh, Thair Hamtini, and Ibrahim Aljarah. Mining educational data to predict student's academic performance using ensemble methods. *International Journal of Database Theory and Application*, 9(8) :119–136, 2016.
- [3] Dagster Labs. Dagster documentation, 2024.
- [4] dbt Labs. dbt documentation, 2024.
- [5] dltHub. dlt documentation, 2024.
- [6] Docker Inc. Docker documentation, 2024.
- [7] DuckDB Foundation. Duckdb documentation, 2024.
- [8] MLflow Team. MLflow documentation, 2024.
- [9] Sebastián Ramírez. Fastapi documentation, 2024.
- [10] Cristobal Romero and Sebastian Ventura. Educational data mining and learning analytics : An updated survey. *WIREs Data Mining and Knowledge Discovery*, 10(3), 2020.
- [11] Ken Schwaber and Jeff Sutherland. *The Scrum Guide*. Scrum.org, 2020.
- [12] Mark Treveil, Nicolas Omont, Clément Stenac, Kenji Lefevre, Du Phan, Joachim Zentici, Adrien Lavoillotte, Makoto Miyazaki, and Léo Heidmann. *Introducing MLOps : How to Scale Machine Learning in the Enterprise*. O'Reilly Media, 2020.